



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Lic. en Sistemas de Información

TP N°2.1 - HTTP - Apache _ Nginx

Redes de Datos

2026

Alumno: Orban, Tobias Naim

DNI: 46.385.637

LU: 58009

1. Ejercicio 1 — Objetivo y alcance del trabajo.....	6
2. Entorno de trabajo utilizado.....	6
2.1. Equipo anfitrión (host).....	6
2.2. Máquina virtual (servidor web).....	6
Ventana del Administrador de Oracle VM VirtualBox mostrando la máquina Ubuntu-Redes con el resumen de configuración: memoria base, almacenamiento y adaptador de red en modo puente.....	7
3. Punto 1 — Acceso al servidor mediante SSH.....	8
3.1. Reconocimiento previo del entorno.....	8
Salida por consola de los comandos <code>lsb_release -a</code> , <code>ip a</code> y <code>systemd-detect-virt</code> , donde se observen la versión de Ubuntu, la dirección IP 192.168.1.2 sobre <code>enp0s3</code> y la respuesta "oracle"	8
3.2. Instalación y habilitación del servidor SSH.....	9
Salida del comando <code>sudo systemctl status ssh</code> mostrando el servicio en estado active (running).....	9
3.3. Conexión remota desde el cliente PuTTY.....	9
Ventana de PuTTY Configuration con la IP 192.168.1.2 y el puerto 22 cargados.....	10
Sesión SSH ya establecida en PuTTY: banner de bienvenida de Ubuntu, prompt <code>redes@ubuntu-server</code> y salida del comando <code>hostname -l</code>	10
4. Punto 2 — Instalación del servidor Apache.....	11
4.1. Comandos utilizados.....	11
4.2. Verificación del servicio.....	11
4.3. Reglas de firewall.....	11
4.4. Verificación desde el navegador.....	12
Salida de <code>systemctl status apache2</code> en estado active (running) junto con la versión devuelta por <code>apache2</code> <code>-V</code>	12
Navegador del host mostrando la página "Apache2 Ubuntu Default Page — It works!" con la URL <code>http://192.168.1.2</code> visible en la barra de direcciones.....	12
4.5. Respuesta al requerimiento: ¿host físico o máquina virtual?.....	13
5. Punto 3 — Creación y parametrización de VirtualHosts.....	13
5.1. Estructura de directorios.....	13
5.2. Contenido de los sitios.....	13
5.3. Archivo de configuración del sitio principal.....	14
5.4. Archivo de configuración del segundo sitio.....	14
5.5. Habilitación de los sitios.....	15
5.6. Resolución de nombres y verificación.....	15
Terminal mostrando el resultado de <code>apache2ctl configtest</code> ("Syntax OK") y el listado de <code>/etc/apache2/sites-enabled/</code> con los enlaces simbólicos de ambos sitios.....	15
Navegador mostrando <code>http://wwwredes.local</code> con la página del sitio principal.....	16
Navegador mostrando <code>http://sitio2.local</code> con el contenido del segundo sitio, evidenciando que ambos coexisten en el mismo servidor.....	16
6. Punto 6 — Publicación de una página HTML en el puerto 80.....	16
Navegador del host accediendo a <code>http://192.168.1.2</code> y mostrando la página propia "Servidor Apache - Redes de Datos 2026"	16
7. Punto 7 — Publicación del directorio /admin en el puerto 80.....	17
7.1. Estrategia de implementación.....	17
7.2. Creación del archivo.....	17
7.3. Verificación.....	18
8. Punto 5 — Parametrización y permisos mediante .htaccess.....	19
8.1. Requisito previo: AllowOverride.....	19
8.2. Creación del archivo de credenciales.....	19
8.3. Contenido del archivo .htaccess.....	19

8.4. Verificación del funcionamiento.....	20
Cuadro de diálogo de autenticación del navegador solicitando usuario y contraseña al acceder a http://192.168.1.2/admin/, con la leyenda del realm visible.....	20
Acceso concedido tras ingresar las credenciales correctas: la página de login de la cátedra visible con la LU del alumno.....	21
9. Punto 4 — Localización e interpretación de los logs.....	22
9.1. Ubicación de los archivos de registro.....	22
9.2. Comandos utilizados para la consulta.....	22
9.3. Formato de registro utilizado.....	22
9.4. Análisis de una entrada del registro de accesos.....	22
9.5. Otros códigos de estado registrados durante las pruebas.....	23
9.6. Análisis del registro de errores.....	24
9.7. Importancia de los registros.....	24
Listado del directorio /var/log/apache2/ mostrando los archivos globales y los específicos de cada VirtualHost.....	25
Contenido de wwwredes_access.log.....	25
Contenido de wwwredes_error.log.....	25
10. Conclusiones.....	26
11. Anexo — Comandos utilizados.....	26
12. Ejercicio 2 — Objetivo y alcance.....	28
13. Puntos 1 y 2 — Configuración del sitio en el puerto 8080.....	28
13.1. Análisis previo: el rol de ports.conf.....	28
13.2. Modificación del archivo ports.conf.....	28
Contenido del archivo /etc/apache2/ports.conf obtenido con cat, donde se observen ambas directivas: Listen 80 y Listen 8080.....	29
13.3. Creación del directorio y del contenido HTML.....	29
13.4. Creación del VirtualHost.....	29
Contenido del archivo /etc/apache2/sites-available/sitio8080.conf obtenido con cat, mostrando el bloque VirtualHost completo con el puerto 8080.....	30
13.5. Habilitación del sitio y validación de la configuración.....	30
Terminal mostrando la salida de a2ensite sitio8080.conf y el resultado "Syntax OK" de apache2ctl configtest.....	31
13.6. Regla de firewall.....	31
14. Punto 3 — Modificaciones necesarias y verificación.....	31
14.1. Síntesis de las modificaciones realizadas.....	31
14.2. Verificación de los puertos en escucha.....	32
Salida del comando sudo ss -tlnp mostrando Apache en escucha simultánea sobre los puertos 80 y 8080 con los mismos PIDs.....	32
14.3. Verificación mediante curl.....	33
Salida del comando curl -I http://localhost:8080 mostrando la respuesta HTTP/1.1 200 OK con todos sus encabezados.....	34
14.4. Verificación desde el navegador del equipo anfitrión.....	34
Navegador del equipo anfitrión accediendo a http://192.168.1.2:8080, con la URL visible en la barra de direcciones y el título "Mi Sitio WEB - Puerto 8080" en la pestaña.....	34
Comparativa en dos pestañas del navegador: http://192.168.1.2 (sitio del Ejercicio 1) y http://192.168.1.2:8080 (sitio del Ejercicio 2), evidenciando la coexistencia de ambos.....	35
15. Registros generados por el nuevo sitio.....	35
Contenido del archivo sitio8080_access.log mostrando las peticiones al nuevo sitio con sus códigos de estado.....	35
16. Conclusiones del Ejercicio 2.....	36

17. Anexo — Comandos utilizados en el Ejercicio 2.....	37
18. Ejercicio 3 — Objetivo y alcance.....	38
19. Punto 1 — Análisis de encabezados HTTP mediante curl.....	38
19.1. Consideración previa sobre la ejecución en Windows.....	38
19.2. Respuesta obtenida con curl -i.....	38
Salida completa del comando curl.exe -i http://192.168.1.2 en el Símbolo del sistema, mostrando encabezados y cuerpo HTML.....	39
19.3. Identificación y explicación de los campos solicitados.....	39
19.4. Encabezados adicionales observados.....	40
19.5. Diferencia respecto de la información mostrada por el navegador.....	40
Herramientas de desarrollador del navegador (F12), pestaña Red, mostrando la petición a 192.168.1.2..	41
19.6. Respuesta ante un recurso inexistente.....	41
Ejecución consecutiva de curl -I y curl -i sobre http://192.168.1.2/noexiste.html, evidenciando el mismo código 404 con y sin cuerpo de respuesta.....	43
20. Punto 2 — Visualización y análisis de los logs.....	43
20.1. Consulta del registro.....	43
Salida de sudo tail -n 20 /var/log/apache2/wwwredes_access.log mostrando las tres peticiones realizadas con curl junto al tráfico previo del navegador.....	44
20.2. Análisis de una entrada del registro.....	44
20.3. Observaciones derivadas del análisis.....	44
Salida del comando grep LogFormat /etc/apache2/apache2.conf, donde se observa que la definición del formato combined utiliza la directiva %O.....	45
21. Conclusiones del Ejercicio 3.....	46
22. Anexo — Comandos utilizados en el Ejercicio 3.....	46
23. Ejercicio 4 — Objetivo y consideración previa.....	47
23.1. Conflicto de puertos entre Apache y Nginx.....	47
24. Punto 1 — Instalación del servidor Nginx.....	47
24.1. Liberación del puerto 80.....	47
Salida de systemctl status apache2 en estado inactive (dead) y del comando ss confirmando que ningún proceso ocupa el puerto 80.....	48
24.2. Comando de instalación.....	48
24.3. Verificación del servicio.....	48
Salida de systemctl status nginx en estado active (running) junto con la versión devuelta por nginx -v.....	48
25. Punto 2 — Creación y habilitación del Server Block.....	48
25.1. Directorio y contenido del sitio.....	48
25.2. Definición del Server Block.....	49
Contenido del archivo /etc/nginx/sites-available/wwwredesnginx obtenido con cat.....	50
25.3. Habilitación mediante enlace simbólico.....	50
Salida del comando ls -l /etc/nginx/sites-enabled/ mostrando el enlace simbólico wwwredesnginx apuntando al archivo de sites-available.....	51
25.4. Validación y aplicación de la configuración.....	51
Terminal mostrando el resultado de nginx -t con los mensajes "syntax is ok" y "test is successful".....	51
Navegador mostrando http://192.168.1.2 con la página "Bienvenido a Nginx - Redes de Datos LU: 58009".....	51
25.5. Comparación entre Apache y Nginx.....	51
26. Punto 3 — Consumo de una API pública externa.....	52
26.1. Modificación del documento HTML.....	52
26.2. Incidencia detectada durante la implementación.....	52
26.3. Verificación y análisis del resultado.....	53

Navegador mostrando la página con el resultado "La IP Pública del cliente es: ..." junto al panel de red de las herramientas de desarrollador..... 54

27. Conclusiones del Ejercicio 4..... 54

28. Anexo — Comandos utilizados en el Ejercicio 4.....55

1. Ejercicio 1 — Objetivo y alcance del trabajo

El presente informe documenta la resolución del Ejercicio 1 del Trabajo Práctico N.º 2.1 de la asignatura Redes de Datos. El objetivo del ejercicio consiste en instalar, desplegar y parametrizar un servidor web Apache HTTP Server sobre una distribución Linux, accediendo al mismo de forma remota mediante el protocolo SSH.

Las tareas abordadas comprenden: el acceso remoto al servidor mediante cliente Putty, la instalación del servicio Apache, la creación y parametrización de VirtualHosts para el alojamiento de múltiples sitios web sobre una misma máquina, la localización e interpretación de los archivos de registro (logs) generados por el servidor, la definición de permisos y restricciones de acceso mediante archivos .htaccess, y la publicación de contenido HTML en el puerto 80, incluyendo un directorio /admin con una página de inicio de sesión provista por la cátedra.

Cada sección incluye los comandos ejecutados, los archivos de configuración utilizados, la fundamentación técnica de las decisiones tomadas y las verificaciones realizadas para comprobar el correcto funcionamiento de cada componente.

2. Entorno de trabajo utilizado

La totalidad del ejercicio se resolvió sobre una máquina virtual, no sobre un host físico. A continuación se detallan las características del entorno, tanto del equipo anfitrión (host) como de la máquina virtual (guest) que cumple el rol de servidor web.

2.1. Equipo anfitrión (host)

Componente	Detalle
Sistema operativo	Microsoft Windows 11
Software de virtualización	Oracle VM VirtualBox
Cliente SSH	PuTTY
Navegador utilizado para las pruebas	Microsoft Edge / Google Chrome
Dirección IP del host en la LAN	192.168.1.31

2.2. Máquina virtual (servidor web)

Parámetro	Valor configurado
Nombre de la VM	Ubuntu-Redes
Sistema operativo	Ubuntu Server 22.04 LTS (Jammy Jellyfish), 64 bits
Memoria RAM asignada	2048 MB
Disco duro virtual	20,00 GB (Ubuntu-Redes.vdi, SATA, dinámico)
Procesadores	1 CPU
Adaptador de red	Adaptador 1 — Intel PRO/1000 MT Desktop en modo Adaptador puente (Bridge)
Interfaz de red del guest	enp0s3

Parámetro	Valor configurado
Dirección MAC	08:00:27:72:3E:9B
Dirección IP asignada	192.168.1.2/24 (por DHCP)
Nombre de host	ubuntu-server
Usuario del sistema	redes

Justificación del modo de red: se seleccionó el modo **Adaptador puente** porque coloca a la máquina virtual como un equipo más dentro de la red física local, obteniendo una dirección IP del mismo rango que el host (192.168.1.0/24). Esto permite que el equipo anfitrión acceda al servidor tanto por SSH (puerto 22) como por HTTP (puerto 80) sin necesidad de configurar reglas de reenvío de puertos, condición indispensable para las verificaciones solicitadas por la cátedra. En modo NAT, en cambio, la VM quedaría aislada detrás de una red privada de VirtualBox y no sería alcanzable directamente desde el navegador del host.



Ventana del Administrador de Oracle VM VirtualBox mostrando la máquina Ubuntu-Redes con el resumen de configuración: memoria base, almacenamiento y adaptador de red en modo puente.

3. Punto 1 — Acceso al servidor mediante SSH

Se solicita acceder al servidor a través de SSH desde una terminal, para realizar la totalidad de la configuración de forma remota. Para ello fue necesario, en primera instancia, habilitar el servicio de servidor SSH dentro de la máquina virtual, ya que la instalación mínima de Ubuntu Server no siempre lo incorpora activo por defecto.

3.1. Reconocimiento previo del entorno

Antes de instalar cualquier servicio se verificó la versión del sistema operativo, la configuración de red de la interfaz y se confirmó que el sistema se ejecuta efectivamente sobre un hipervisor. Los comandos utilizados fueron:

```
lsb_release -a
ip a
systemd-detect-virt
```

Interpretación de los resultados obtenidos:

- `lsb_release -a` informó Ubuntu 22.04 LTS (codename jammy), cumpliendo con el requerimiento de la cátedra.
- `ip a` mostró la interfaz `enp0s3` en estado UP, con la dirección `inet 192.168.1.2/24` obtenida dinámicamente y la dirección física `08:00:27:72:3e:9b`. El prefijo `08:00:27` corresponde al OUI asignado a Oracle VirtualBox, lo que constituye una evidencia adicional de que se trata de una máquina virtual.
- `systemd-detect-virt` devolvió `oracle`, confirmando de manera inequívoca que el sistema se ejecuta como guest bajo el hipervisor de VirtualBox y no sobre hardware físico.

```
redes@ubuntu-server:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 22.04.4 LTS
Release:      22.04
Codename:     jammy
redes@ubuntu-server:~$ systemd-detect-virt
oracle
redes@ubuntu-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:72:3e:9b brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.2/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 83860sec preferred_lft 83860sec
    inet6 fe80::a00:27ff:fe72:3e9b/64 scope link
        valid_lft forever preferred_lft forever
```

Salida por consola de los comandos `lsb_release -a`, `ip a` y `systemd-detect-virt`, donde se observen la versión de Ubuntu, la dirección IP 192.168.1.2 sobre `enp0s3` y la respuesta "oracle".

3.2. Instalación y habilitación del servidor SSH

Se actualizó el índice de paquetes del sistema y se instaló el paquete openssh-server, que provee el demonio sshd. Posteriormente se habilitó el servicio para que inicie automáticamente con el sistema y se lo arrancó en la misma operación mediante la opción --now:

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo systemctl status ssh
sudo ufw allow OpenSSH
```

El comando `systemctl enable --now ssh` generó el enlace simbólico de arranque automático y dejó el servicio en estado active (running). Con `sudo ufw allow OpenSSH` se cargó la regla de firewall correspondiente al puerto 22/TCP. Cabe aclarar que, tal como se detalla en la sección 4.3, el firewall UFW se encuentra en estado inactivo en esta instalación, por lo que las reglas quedan precargadas y se aplicarían recién al habilitarlo.

Adicionalmente se comprobó que el demonio se encontrara efectivamente escuchando en el puerto 22 en todas las interfaces:

```
ss -tlnp | grep :22
```

```
redes@ubuntu-server:~$ sudo systemctl status ssh
[sudo] password for redes:
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-09-04 11:01:32 UTC; 37min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
  Main PID: 1886 (sshd)
    Tasks: 1 (limit: 2216)
   Memory: 4.0M
      CPU: 131ms
   CGroup: /system.slice/ssh.service
           └─1886 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

sep 04 11:01:31 ubuntu-server systemd[1]: Starting OpenBSD Secure Shell server...
sep 04 11:01:32 ubuntu-server sshd[1886]: Server listening on 0.0.0.0 port 22.
sep 04 11:01:32 ubuntu-server sshd[1886]: Server listening on :: port 22.
sep 04 11:01:32 ubuntu-server systemd[1]: Started OpenBSD Secure Shell server.
sep 04 11:05:00 ubuntu-server sshd[2208]: Accepted password for redes from 192.168.1.31 port 50087 >
sep 04 11:05:00 ubuntu-server sshd[2208]: pam_unix(sshd:session): session opened for user redes(uid>
lines 1-18/18 (END)
```

Salida del comando `sudo systemctl status ssh` mostrando el servicio en estado active (running).

3.3. Conexión remota desde el cliente PuTTY

Desde el equipo anfitrión Windows se utilizó el cliente PuTTY para establecer la sesión remota. Los parámetros de conexión configurados fueron:

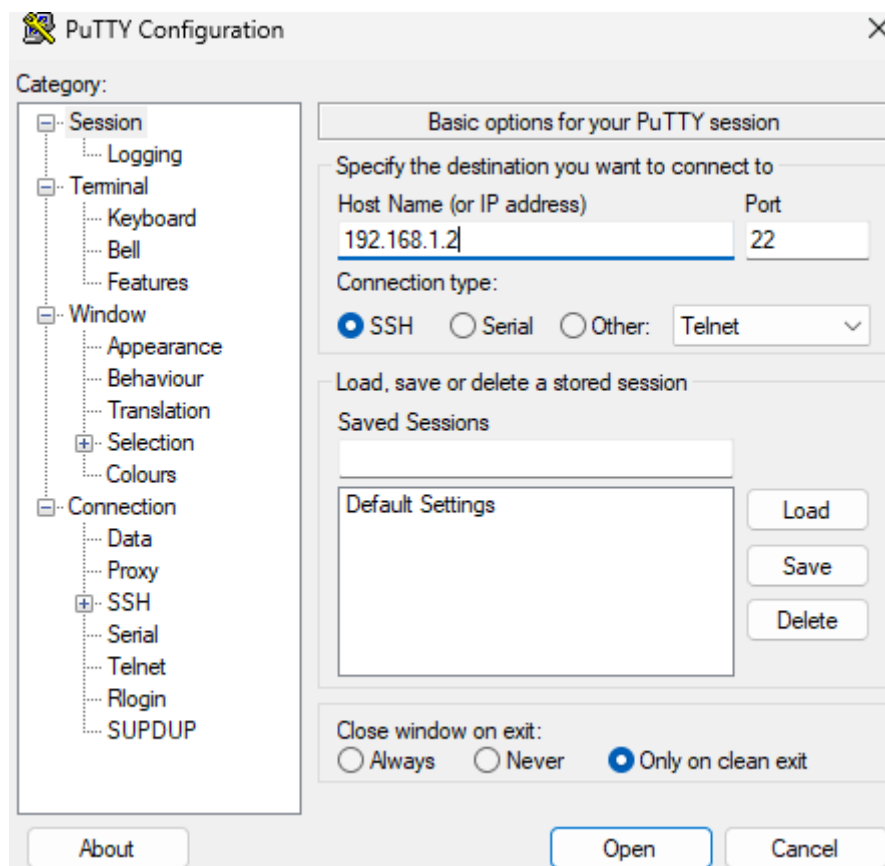
Campo de PuTTY	Valor ingresado
Host Name (or IP address)	192.168.1.2
Port	22
Connection type	SSH
Saved Session	Ubuntu-Redes
Usuario de acceso	redes

Al establecer la conexión por primera vez, PuTTY presentó una alerta de seguridad mostrando la huella digital (fingerprint) de la clave pública del servidor. Esta advertencia es el mecanismo de confianza en el primer uso (Trust On First Use) del protocolo SSH: dado que el cliente no conoce previamente la clave del host, solicita al usuario que la valide manualmente. Una vez aceptada, la clave queda almacenada localmente y cualquier cambio posterior en la misma generaría una advertencia de posible suplantación del servidor.

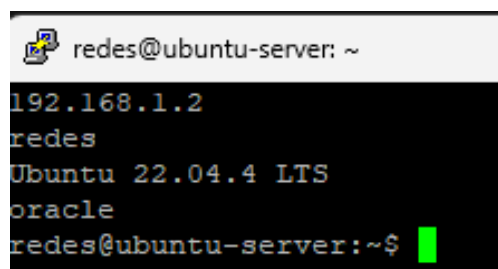
Aceptada la clave, se ingresaron las credenciales del usuario. Se destaca que, por diseño del protocolo, la contraseña no muestra ningún carácter en pantalla mientras se escribe. Una vez autenticado, se obtuvo el prompt del sistema y se verificó la identidad del equipo y del usuario con:

```
hostname -I && whoami
```

La salida confirmó la dirección 192.168.1.2 y el usuario redes, quedando establecida la sesión remota. A partir de este punto, la totalidad de la configuración del servidor Apache se realizó desde esta terminal SSH.



Ventana de PuTTY Configuration con la IP 192.168.1.2 y el puerto 22 cargados.



Sesión SSH ya establecida en PuTTY: banner de bienvenida de Ubuntu, prompt redes@ubuntu-server y salida del comando hostname -I.

4. Punto 2 — Instalación del servidor Apache

4.1. Comandos utilizados

La instalación se realizó desde los repositorios oficiales de Ubuntu mediante el gestor de paquetes APT, ejecutando los siguientes comandos a través de la sesión SSH previamente establecida:

```
sudo apt update
sudo apt install -y apache2
```

El primer comando actualiza la lista de paquetes disponibles desde los repositorios configurados; el segundo instala el paquete apache2 junto con sus dependencias. La opción `-y` confirma automáticamente la instalación sin requerir interacción del usuario.

Durante el proceso, el propio instalador informó la creación de los enlaces simbólicos de arranque del servicio y la habilitación del sitio por defecto, mediante mensajes como "Enabling site 000-default" y "Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service", lo que indica que el servicio quedó configurado para iniciarse automáticamente con el sistema.

4.2. Verificación del servicio

Se comprobó el estado del servicio y la versión instalada:

```
sudo systemctl status apache2
apache2 -v
```

Los resultados obtenidos fueron:

Parámetro verificado	Resultado
Estado del servicio	active (running)
Arranque automático	enabled
PID principal	3244 (apache2), con procesos hijos 3246 y 3247
Versión del servidor	Apache/2.4.52 (Ubuntu)
Fecha de compilación	2026-07-06T16:41:46

Se observa además el mensaje AH00558: apache2: Could not reliably determine the server's fully qualified domain name. Se trata de una advertencia y no de un error: Apache no encuentra una directiva `ServerName` global, por lo que utiliza la dirección IP local. Esta advertencia desaparece al definir la directiva `ServerName` dentro de los `VirtualHosts`, tal como se realizó en el punto siguiente.

4.3. Reglas de firewall

Se cargó el perfil de aplicación de Apache en el firewall y se consultó su estado:

```
sudo ufw allow 'Apache Full'
sudo ufw status
```

El perfil `Apache Full` habilita los puertos 80/TCP (HTTP) y 443/TCP (HTTPS). La consulta de estado devolvió `Status: inactive`, es decir que UFW se encuentra deshabilitado en esta instalación. En consecuencia, el sistema no está filtrando tráfico y las reglas quedan almacenadas, aplicándose únicamente si se activara el firewall mediante `sudo ufw enable`. Se decidió no habilitarlo durante el desarrollo del práctico para evitar una eventual interrupción de la propia sesión SSH de administración.

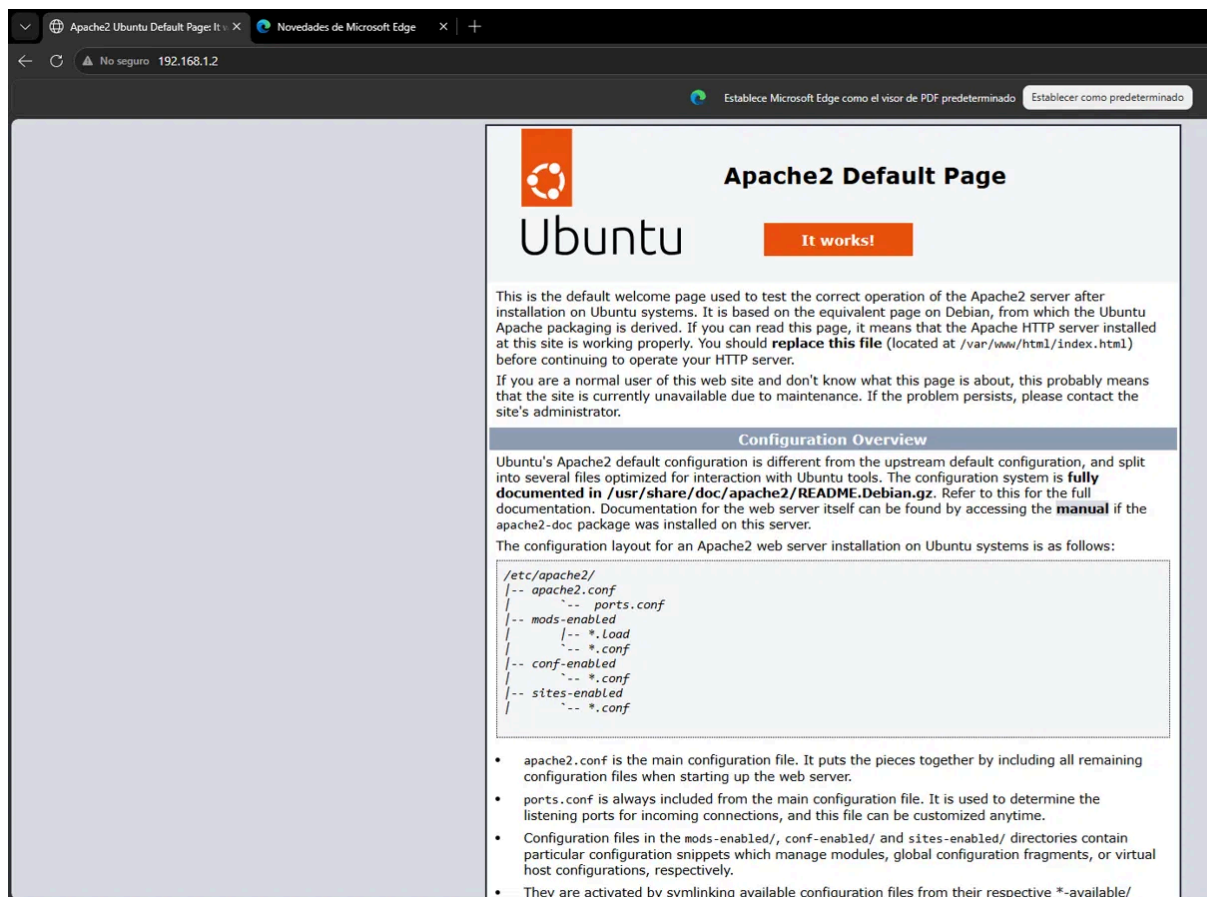
4.4. Verificación desde el navegador

Desde el navegador del equipo anfitrión se accedió a `http://192.168.1.2`, obteniéndose la página por defecto "Apache2 Ubuntu Default Page — It works!". Esta página confirma que el servicio responde correctamente peticiones HTTP sobre el puerto 80 y que existe conectividad de red entre el host y la máquina virtual.

```
redes@ubuntu-server:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-09-04 11:07:06 UTC; 37min ago
     Docs: https://httpd.apache.org/docs/2.4/
  Process: 3731 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
 Main PID: 3244 (apache2)
    Tasks: 55 (limit: 2216)
   Memory: 5.9M
      CPU: 512ms
   CGroup: /system.slice/apache2.service
           └─3244 /usr/sbin/apache2 -k start
             └─3736 /usr/sbin/apache2 -k start
               └─3737 /usr/sbin/apache2 -k start

sep 04 11:07:06 ubuntu-server systemd[1]: Starting The Apache HTTP Server...
sep 04 11:07:06 ubuntu-server apachectl[3243]: AH00558: apache2: Could not reliably determine the serv
sep 04 11:07:06 ubuntu-server systemd[1]: Started The Apache HTTP Server.
sep 04 11:10:32 ubuntu-server systemd[1]: Reloading The Apache HTTP Server...
sep 04 11:10:32 ubuntu-server apachectl[3734]: AH00558: apache2: Could not reliably determine the serv
sep 04 11:10:32 ubuntu-server systemd[1]: Reloaded The Apache HTTP Server.
```

Salida de `systemctl status apache2` en estado active (running) junto con la versión devuelta por `apache2 -v`.



Navegador del host mostrando la página "Apache2 Ubuntu Default Page — It works!" con la URL `http://192.168.1.2` visible en la barra de direcciones.

4.5. Respuesta al requerimiento: ¿host físico o máquina virtual?

El servidor Apache fue instalado sobre una **máquina virtual**, no sobre un host físico. Esta afirmación se sustenta en tres evidencias independientes: (1) la salida del comando `systemd-detect-virt`, que devuelve `oracle`; (2) la dirección MAC de la interfaz `enp0s3`, cuyo prefijo `08:00:27` pertenece al rango OUI registrado por Oracle para VirtualBox; y (3) la propia consola del Administrador de VirtualBox, que muestra la máquina Ubuntu-Redes en ejecución.

5. Punto 3 — Creación y parametrización de VirtualHosts

Un VirtualHost permite que un único servidor Apache, con una sola dirección IP, aloje varios sitios web independientes. Apache distingue a cuál sitio corresponde cada petición mediante el encabezado Host que envía el cliente HTTP/1.1, comparándolo con la directiva `ServerName` de cada VirtualHost definido. A esta modalidad se la denomina VirtualHost basado en nombre (`name-based`).

Para demostrar esta capacidad se configuraron dos sitios independientes sobre el mismo servidor.

5.1. Estructura de directorios

Se crearon los directorios raíz de cada sitio, incluyendo el subdirectorio `/admin` requerido en el punto 7:

```
sudo mkdir -p /var/www/wwwredes/admin
sudo mkdir -p /var/www/sitio2
```

Posteriormente se asignaron el propietario y los permisos correspondientes:

```
sudo chown -R www-data:www-data /var/www/wwwredes /var/www/sitio2
sudo chmod -R 755 /var/www
ls -l /var/www/
```

Fundamentación de los permisos: en Ubuntu, Apache ejecuta sus procesos hijos bajo el usuario y grupo `www-data`. Al asignar la propiedad de los directorios a ese usuario, el servidor puede leer el contenido sin necesidad de privilegios elevados. El modo `755` otorga lectura, escritura y ejecución al propietario, y solo lectura y ejecución al grupo y al resto de los usuarios. El permiso de ejecución sobre un directorio es lo que habilita atravesarlo para alcanzar los archivos que contiene; sin él, Apache devolvería un error 403 aun cuando el archivo HTML tuviera permisos de lectura.

5.2. Contenido de los sitios

Se generó la página principal del sitio 1, que da cumplimiento al punto 6 del enunciado (publicación de una página HTML en el puerto 80):

```
sudo tee /var/www/wwwredes/index.html > /dev/null <<'EOF'
<!doctype html>
<html lang="es">
<head><meta charset="utf-8"><title>Redes de Datos - Sitio principal</title></head>
<body>
  <h1>Servidor Apache - Redes de Datos 2026</h1>
  <p>Sitio publicado en el puerto 80 - UNNE - Tobias Orban</p>
</body>
</html>
EOF
```

Y una página mínima para el segundo sitio, cuyo único fin es evidenciar que ambos VirtualHosts entregan contenidos distintos:

```
echo '<h1>Sitio 2 - VirtualHost por nombre</h1>' | sudo tee /var/www/sitio2/index.html
```

5.3. Archivo de configuración del sitio principal

Se creó el archivo `/etc/apache2/sites-available/000-wwwredes.conf` con el siguiente contenido:

```
<VirtualHost *:80>
    ServerName wwwredes.local
    ServerAdmin admin@wwwredes.local
    DocumentRoot /var/www/wwwredes

    <Directory /var/www/wwwredes>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/wwwredes_error.log
    CustomLog ${APACHE_LOG_DIR}/wwwredes_access.log combined
</VirtualHost>
```

Significado de cada directiva utilizada:

Directiva	Función
<VirtualHost *:80>	Define el host virtual para peticiones recibidas en cualquier dirección IP del servidor sobre el puerto 80.
ServerName	Nombre de dominio que identifica al sitio. Apache lo compara con el encabezado Host de la petición.
ServerAdmin	Dirección de correo del administrador, mostrada en las páginas de error generadas por el servidor.
DocumentRoot	Directorio del sistema de archivos desde el cual se sirve el contenido de este sitio.
Options -Indexes	Deshabilita el listado automático del contenido del directorio cuando no existe un archivo índice.
Options +FollowSymLinks	Permite que Apache siga enlaces simbólicos dentro del directorio.
AllowOverride All	Habilita que los archivos <code>.htaccess</code> de este árbol puedan sobrescribir la configuración. Sin esta directiva el <code>.htaccess</code> sería ignorado.
Require all granted	Concede acceso a todos los clientes (control de acceso de Apache 2.4).
ErrorLog	Ruta del archivo de registro de errores propio de este VirtualHost.
CustomLog ... combined	Ruta del registro de accesos y formato de log utilizado (combined incluye referer y user-agent).

5.4. Archivo de configuración del segundo sitio

De manera análoga se creó `/etc/apache2/sites-available/sitio2.conf`:

```
<VirtualHost *:80>
    ServerName sitio2.local
    DocumentRoot /var/www/sitio2

    <Directory /var/www/sitio2>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
```

```
ErrorLog    ${APACHE_LOG_DIR}/sitio2_error.log
CustomLog  ${APACHE_LOG_DIR}/sitio2_access.log combined
</VirtualHost>
```

5.5. Habilitación de los sitios

Apache en Ubuntu separa las configuraciones disponibles de las activas mediante dos directorios: sites-available contiene todos los archivos de configuración definidos, mientras que sites-enabled contiene enlaces simbólicos hacia aquellos que efectivamente se cargan. Las herramientas a2ensite y a2dissite administran esos enlaces:

```
sudo a2ensite 000-wwwredes.conf
sudo a2ensite sitio2.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
ls -l /etc/apache2/sites-enabled/
```

Se deshabilitó el sitio por defecto 000-default.conf para que las peticiones dirigidas a la IP del servidor sean atendidas por el sitio propio y no por la página de bienvenida de Ubuntu. El comando apache2ctl configtest validó la sintaxis de la configuración antes de aplicarla, devolviendo Syntax OK. Finalmente, systemctl reload apache2 recargó la configuración sin interrumpir las conexiones en curso, a diferencia de restart que reinicia por completo el servicio.

Criterio de nomenclatura: Apache procesa los archivos de sites-enabled en orden alfabético. El primer VirtualHost que coincide con la combinación IP:puerto actúa como sitio por defecto cuando la petición no incluye un encabezado Host coincidente con ningún ServerName, como ocurre al acceder directamente por dirección IP. Por ese motivo el sitio principal se nombró con el prefijo 000-, garantizando que http://192.168.1.2 resuelva hacia wwwredes y no hacia sitio2.

5.6. Resolución de nombres y verificación

Dado que los nombres wwwredes.local y sitio2.local no existen en ningún servidor DNS, se los resolvió localmente agregando las siguientes entradas al archivo de hosts del equipo anfitrión (C:\Windows\System32\drivers\etc\hosts), editado con privilegios de administrador:

```
192.168.1.2    wwwredes.local
192.168.1.2    sitio2.local
```

Con esta configuración se verificó desde el navegador que ambos nombres, apuntando a la misma dirección IP y al mismo puerto, entregan contenidos distintos, comprobando así el funcionamiento del mecanismo de VirtualHosts basados en nombre.

```
redes@ubuntu-server:~$ apache2ctl
Usage: /usr/sbin/apache2ctl start|stop|restart|graceful|graceful-stop|configtest|status|fullstatus|help
       /usr/sbin/apache2ctl <apache2 args>
       /usr/sbin/apache2ctl -h           (for help on <apache2 args>)
```

Terminal mostrando el resultado de apache2ctl configtest ("Syntax OK") y el listado de /etc/apache2/sites-enabled/ con los enlaces simbólicos de ambos sitios.



Navegador mostrando <http://wwwredes.local> con la página del sitio principal.



Navegador mostrando <http://sitio2.local> con el contenido del segundo sitio, evidenciando que ambos coexisten en el mismo servidor.

6. Punto 6 — Publicación de una página HTML en el puerto 80

El requerimiento se satisface con el sitio principal definido en la sección anterior. El archivo `/var/www/wwwredes/index.html` es servido por el VirtualHost `000-wwwredes.conf`, que escucha en el puerto 80. El puerto de escucha está declarado globalmente en el archivo `/etc/apache2/ports.conf` mediante la directiva `Listen 80`, y es referenciado por cada VirtualHost en su encabezado `<VirtualHost *:80>`.

La verificación se realizó accediendo desde el navegador del equipo anfitrión a la dirección IP del servidor. La página por defecto de Ubuntu fue reemplazada por el contenido propio, confirmando que el VirtualHost tomó efecto correctamente.



Navegador del host accediendo a <http://192.168.1.2> y mostrando la página propia "Servidor Apache - Redes de Datos 2026".

7. Punto 7 — Publicación del directorio /admin en el puerto 80

Se solicitó publicar, dentro del puerto 80, el directorio /admin conteniendo una página HTML de inicio de sesión provista por la cátedra, reemplazando el marcador [XXXXXX] por la Libreta Universitaria del alumno.

7.1. Estrategia de implementación

Se optó por crear el directorio como subdirectorio del DocumentRoot del sitio principal (/var/www/wwwredes/admin). De esta forma, la ruta `http://192.168.1.2/admin/` se resuelve de manera directa por correspondencia entre la URL y el sistema de archivos, sin necesidad de directivas adicionales.

Alternativa considerada: de haberse requerido que el contenido residiera fuera del DocumentRoot, la solución habría sido incorporar dentro del VirtualHost una directiva `Alias /admin /ruta/externa` acompañada de su correspondiente bloque `<Directory>` con los permisos de acceso. Se descartó esta opción por resultar innecesaria para el alcance del ejercicio.

7.2. Creación del archivo

El archivo se editó mediante el editor nano a través de la sesión SSH, incorporando íntegramente el código HTML provisto en el enunciado y reemplazando el marcador correspondiente por la LU del alumno:

```
sudo nano /var/www/wwwredes/admin/index.html
```

El documento contiene un formulario de inicio de sesión con campos de usuario y contraseña, estilado mediante CSS embebido, y un encabezado con la leyenda "Redes de Datos | LU:" seguida del número de libreta.

```
redes@ubuntu-server:~$ cat /var/www/wwwredes/admin/index.html
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Redes de Datos </title>
  <style>
    body { font-family: system-ui, sans-serif; background: #f1f5f9; display: grid; place-items: center;
min-height: 100vh; margin: 0; }
    #container { background: #fff; padding: 2rem; border-radius: 12px; box-shadow: 0 4px 12px rgba(0,0,
0,0.1); width: 280px; }
    h1 { font-size: 1.4rem; text-align: center; margin-top: 0; color: #0f172a; }
    label { font-size: 0.85rem; font-weight: 600; display: block; margin: 0.8rem 0 0.2rem; }
    input { width: 100%; padding: 0.6rem; border: 1px solid #cbd5e1; border-radius: 6px; box-sizing: bo
rder-box; }
    input[type="submit"] { background: #2563eb; color: #fff; border: none; font-weight: 600; margin-top
: 1.2rem; cursor: pointer; }
    input[type="submit"]:hover { background: #1d4ed8; }
  </style>
</head>
<body>
  <div id="container">
    <h1>Redes de Datos | LU: 58009</h1>
    <form>
      <label for="user">Usuario:</label>
      <input type="text" id="user" required>
      <label for="pass">Contraseña:</label>
      <input type="password" id="pass" required>
      <input type="submit" value="Login">
    </form>
  </div>
</body>
</html>
```

A continuación se ajustaron el propietario y los permisos del contenido:

```
sudo chown -R www-data:www-data /var/www/wwwredes/admin
sudo chmod 644 /var/www/wwwredes/admin/index.html
```

Se asignó el modo 644 al archivo (lectura y escritura para el propietario, solo lectura para el resto), que es el permiso adecuado para contenido estático: Apache únicamente necesita leerlo, no ejecutarlo.

7.3. Verificación

Se accedió desde el navegador del host a `http://192.168.1.2/admin/`, obteniéndose correctamente la página de inicio de sesión con la LU visible en el encabezado.

Observación registrada: al solicitar la ruta sin la barra final (`/admin`), el servidor respondió con un código 301 Moved Permanently redirigiendo hacia `/admin/`. Este comportamiento lo genera el módulo `mod_dir` y es necesario para que las rutas relativas contenidas en la página se resuelvan correctamente. La evidencia de esta redirección quedó registrada en el log de accesos analizado en la sección 9.

8. Punto 5 — Parametrización y permisos mediante .htaccess

Un archivo .htaccess permite aplicar directivas de configuración a nivel de directorio, sin necesidad de modificar los archivos de configuración principales ni reiniciar el servicio. Su principal ventaja es la flexibilidad para delegar la administración de un subdirectorio; su desventaja es el costo de rendimiento, ya que Apache debe consultar el archivo en cada petición y en todo el árbol de directorios que la contiene.

Se decidió utilizar .htaccess para restringir el acceso al directorio /admin mediante autenticación básica HTTP, dado que se trata de un área de administración.

8.1. Requisito previo: AllowOverride

Para que Apache procese los archivos .htaccess es indispensable que el bloque <Directory> correspondiente incluya la directiva AllowOverride con un valor distinto de None. El valor por defecto en Ubuntu es None, lo que provoca que el archivo sea ignorado silenciosamente, sin generar ningún error visible. Por este motivo, el VirtualHost del sitio principal fue definido desde el inicio con AllowOverride All (ver sección 5.3).

8.2. Creación del archivo de credenciales

Se instaló el paquete de utilidades de Apache, que provee la herramienta htpasswd, y se generó el archivo de credenciales con un usuario administrador:

```
sudo apt install -y apache2-utils
sudo htpasswd -c /etc/apache2/.htpasswd admin
sudo chown root:www-data /etc/apache2/.htpasswd
sudo chmod 640 /etc/apache2/.htpasswd
```

La opción -c crea el archivo; debe utilizarse únicamente la primera vez, ya que sobrescribe el contenido existente. Para agregar usuarios adicionales se omite dicha opción. Las contraseñas se almacenan cifradas, no en texto plano.

Decisiones de seguridad adoptadas: el archivo de credenciales se ubicó en /etc/apache2/ y no dentro del DocumentRoot, de modo que no pueda ser descargado por un cliente HTTP. Además se le asignó el modo 640 con propietario root y grupo www-data: solo root puede modificarlo y solo Apache puede leerlo, quedando inaccesible para el resto de los usuarios del sistema.

8.3. Contenido del archivo .htaccess

Se creó el archivo /var/www/wwwredes/admin/.htaccess con las siguientes directivas:

```
AuthType Basic
AuthName "Area restringida - Redes de Datos"
AuthUserFile /etc/apache2/.htpasswd
Require valid-user

Options -Indexes
```

Directiva	Función
AuthType Basic	Establece el esquema de autenticación HTTP Basic definido en el RFC 7617.
AuthName	Define el "realm" o texto descriptivo que el navegador muestra en el cuadro de diálogo de credenciales.

Directiva	Función
AuthUserFile	Indica la ruta absoluta del archivo que contiene los usuarios y contraseñas cifradas.
Require valid-user	Exige que el cliente se autentique con cualquier usuario válido del archivo de credenciales.
Options -Indexes	Impide el listado del contenido del directorio en caso de ausencia de archivo índice.

Aplicada la configuración, se recargó el servicio:

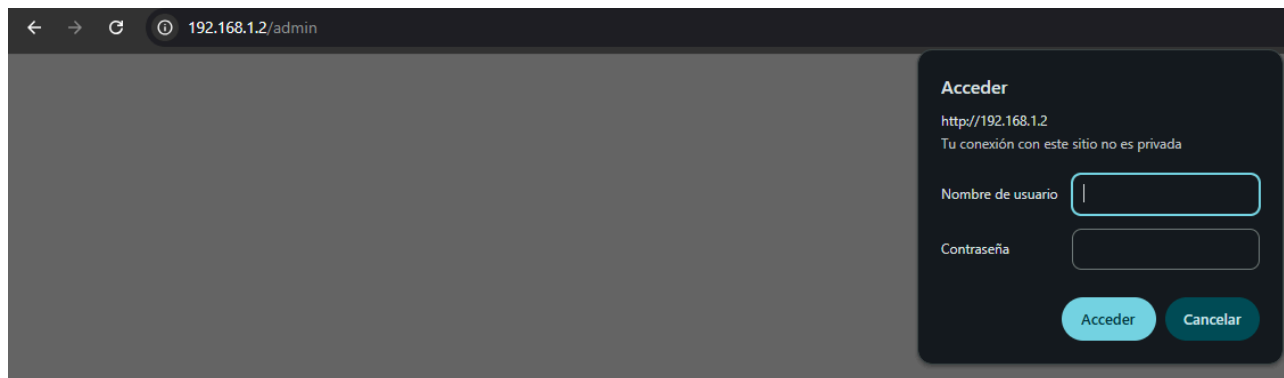
```
sudo systemctl reload apache2
```

8.4. Verificación del funcionamiento

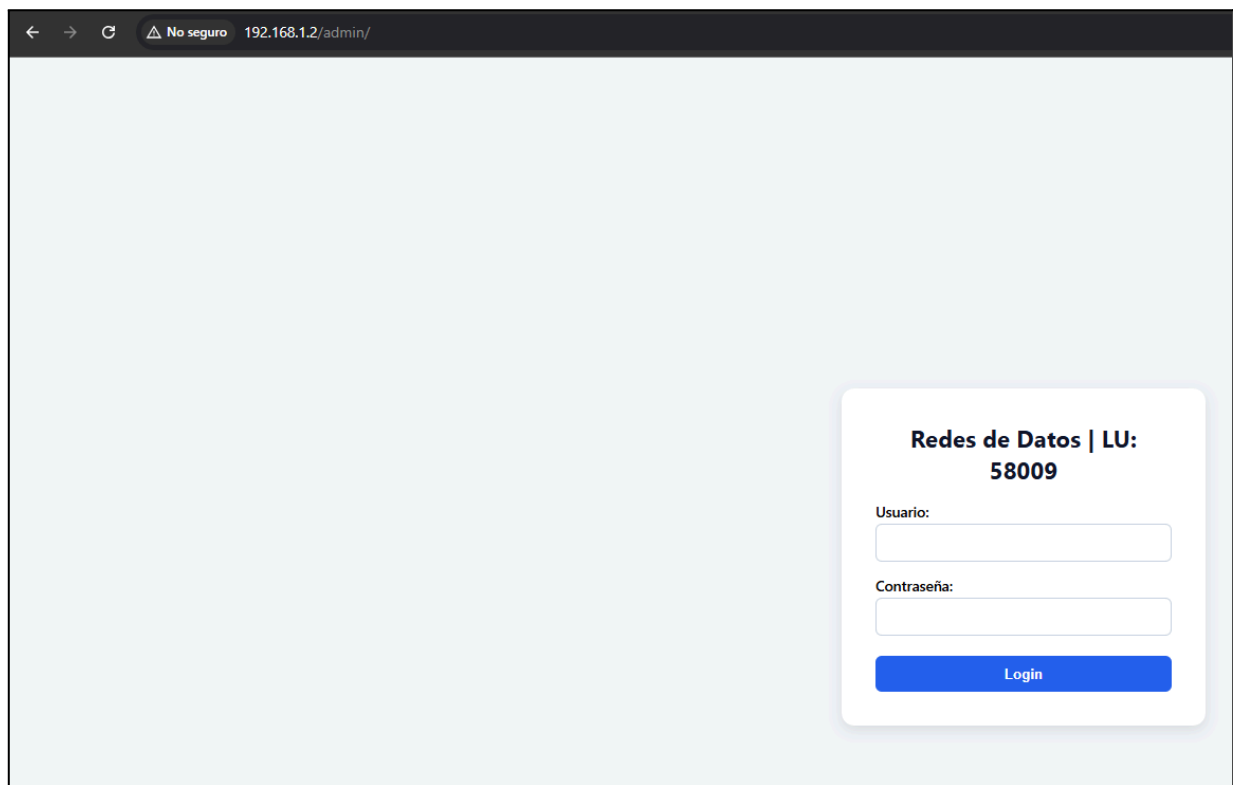
Al intentar acceder nuevamente a `http://192.168.1.2/admin/`, el servidor respondió con un código 401 Unauthorized acompañado del encabezado `WWW-Authenticate`, lo que provocó que el navegador desplegara el cuadro de diálogo solicitando nombre de usuario y contraseña.

Este intercambio ilustra el funcionamiento del esquema de autenticación básica: el cliente realiza una primera petición sin credenciales, el servidor la rechaza indicando el método de autenticación requerido, y el cliente reenvía la misma petición incorporando el encabezado `Authorization`. Por ello, en el registro de accesos se observan dos entradas consecutivas para el mismo recurso: primero un 401 y luego un 200.

Se realizaron dos pruebas complementarias: un intento con un usuario inexistente, que fue rechazado con 401 y dejó constancia en el log de errores, y un acceso con las credenciales correctas, que devolvió 200 y permitió visualizar la página de login solicitada en el punto 7.



Cuadro de diálogo de autenticación del navegador solicitando usuario y contraseña al acceder a `http://192.168.1.2/admin/`, con la leyenda del realm visible.



Acceso concedido tras ingresar las credenciales correctas: la página de login de la cátedra visible con la LU del alumno.

9. Punto 4 — Localización e interpretación de los logs

9.1. Ubicación de los archivos de registro

En Ubuntu, Apache almacena sus registros en el directorio `/var/log/apache2/`. La ruta está parametrizada mediante la variable de entorno `${APACHE_LOG_DIR}`, definida en `/etc/apache2/envvars`. El listado del directorio se obtuvo con:

```
ls -l /var/log/apache2/
```

Los archivos presentes en el servidor y su función son los siguientes:

Archivo	Contenido
access.log	Registro global de accesos generado por la configuración por defecto.
error.log	Registro global de errores del servidor, incluidos los de arranque y sintaxis.
other_vhosts_access.log	Accesos de aquellos VirtualHosts que no definen su propia directiva CustomLog.
wwwredes_access.log	Accesos al sitio principal, generado por la directiva CustomLog del VirtualHost.
wwwredes_error.log	Errores propios del sitio principal, generado por su directiva ErrorLog.
sitio2_access.log	Accesos al segundo sitio.
sitio2_error.log	Errores propios del segundo sitio.

La existencia de archivos separados por sitio es consecuencia directa de haber declarado `ErrorLog` y `CustomLog` dentro de cada `VirtualHost`. Esta separación resulta especialmente valiosa en servidores que alojan múltiples sitios, ya que permite aislar el análisis del tráfico y el diagnóstico de fallas de cada uno de manera independiente.

9.2. Comandos utilizados para la consulta

```
sudo tail -n 20 /var/log/apache2/wwwredes_access.log
sudo tail -n 20 /var/log/apache2/wwwredes_error.log
sudo tail -f /var/log/apache2/wwwredes_access.log
```

La opción `-n 20` muestra las últimas veinte líneas del archivo, mientras que `-f` mantiene la lectura en tiempo real, permitiendo observar cómo se registran las peticiones a medida que se generan desde el navegador.

9.3. Formato de registro utilizado

Los `VirtualHosts` fueron configurados con el formato `combined`, definido en `/etc/apache2/apache2.conf`. Puede consultarse su definición con:

```
grep LogFormat /etc/apache2/apache2.conf
```

Su estructura es la siguiente:

```
%h %l %u %t "%r" %>s %O "%{Referer}i" "%{User-Agent}i"
```

9.4. Análisis de una entrada del registro de accesos

Se selecciona la siguiente entrada real, correspondiente a un acceso exitoso al área administrativa tras autenticarse:

```
192.168.1.31 - admin [04/Sep/2026:11:16:45 +0000] "GET /admin/ HTTP/1.1"
200 954 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36 Edg/152.0.0.0"
```

Descomposición campo por campo:

Campo	Valor	Interpretación
%h — IP cliente	192.168.1.31	Dirección del equipo que originó la petición: el host Windows.
%l — identidad	-	Identidad RFC 1413, en desuso. El guion indica dato no disponible.
%u — usuario	admin	Usuario autenticado por el .htaccess. Solo aparece si hubo autenticación exitosa.
%t — timestamp	[04/Sep/2026:11:16:45 +0000]	Fecha y hora de finalización de la petición, con el desplazamiento respecto de UTC.
%r — línea de petición	GET /admin/ HTTP/1.1	Método HTTP, recurso solicitado y versión del protocolo.
%>s — código de estado	200	Respuesta final del servidor: petición procesada con éxito.
%O — bytes enviados	954	Total de bytes transmitidos al cliente, encabezados incluidos. Ubuntu define el formato <i>combined</i> con %O en lugar de %b, por lo que el valor supera al tamaño del recurso entregado.
%{Referer}i	-	URL de origen del enlace. Ausente por tratarse de un acceso directo.
%{User-Agent}i	Mozilla/5.0 ... Edg/152	Identificación del navegador y del sistema operativo del cliente.

Respondiendo puntualmente a lo solicitado por el enunciado para esta entrada:

- IP del cliente: 192.168.1.31
- Método HTTP: GET
- Recurso solicitado: /admin/
- Código HTTP de respuesta: 200 (OK)

9.5. Otros códigos de estado registrados durante las pruebas

El tráfico generado durante el desarrollo del práctico dejó constancia de diversos códigos de respuesta, cuyo análisis resulta ilustrativo del comportamiento del servidor:

Código	Petición registrada	Causa
301	GET /admin	Redirección permanente hacia /admin/ generada por mod_dir al solicitar un directorio sin barra final.
403	GET /admin/	El directorio aún no contenía index.html y la directiva Options -Indexes impedía listar su contenido.

Código	Petición registrada	Causa
200	GET /	Entrega exitosa de la página principal del sitio.
404	GET /favicon.ico	El navegador solicita automáticamente el icono del sitio, que no fue creado.
401	GET /admin/ (usuario redes)	Credenciales inválidas: el usuario no existe en el archivo .htpasswd.
404	HEAD /noexiste.html	Recurso inexistente solicitado deliberadamente mediante curl para verificar el registro.
408	Conexión sin petición	Request Timeout: conexiones abiertas anticipadamente por el navegador que expiran sin enviar datos.

Resulta relevante señalar que las peticiones originadas en el navegador registran como IP cliente la dirección 192.168.1.31 (equipo anfitrión), mientras que la ejecutada con curl desde la propia máquina virtual registra 192.168.1.2. Esto confirma que el campo %h refleja fielmente el origen real de cada solicitud.

9.6. Análisis del registro de errores

El archivo `wwwredes_error.log` registró las siguientes entradas, que constituyen la contraparte explicativa de los códigos observados en el log de accesos:

```
[autoindex:error] AH01276: Cannot serve directory /var/www/wwwredes/admin/:
No matching DirectoryIndex (index.html, index.cgi, ...) found, and
server-generated directory index forbidden by Options directive

[auth_basic:error] AH01618: user redes not found: /admin/
```

- **AH01276** (módulo `autoindex`) explica el código 403: Apache no encontró ningún archivo índice en el directorio y la directiva `Options -Indexes` le impidió generar un listado automático. El error desapareció al crear `index.html`.
- **AH01618** (módulo `auth_basic`) explica el código 401: se intentó autenticar con el usuario `redes`, inexistente en el archivo de credenciales, en el cual solo fue dado de alta el usuario `admin`.

9.7. Importancia de los registros

El análisis de logs constituye una herramienta fundamental en la administración de servidores web por múltiples razones:

- El registro de accesos permite monitorear el volumen y la naturaleza del tráfico, identificar los recursos más solicitados y detectar patrones anómalos, como intentos repetidos de autenticación fallida o escaneos automatizados de rutas conocidas.
- El registro de errores es el primer punto de consulta ante cualquier falla, ya que documenta la causa técnica precisa de las respuestas de error, información que el cliente nunca recibe.
- La correlación entre ambos archivos permite reconstruir el comportamiento completo de una petición: el `access.log` indica qué respondió el servidor, mientras que el `error.log` indica por qué.
- Desde la perspectiva de la seguridad, los registros conservan la trazabilidad de los accesos a áreas restringidas, incluyendo el usuario autenticado, la dirección de origen y el momento exacto de cada acción.

```

redes@ubuntu-server:~$ ls -l /var/log/apache2/
total 24
-rw-r----- 1 root adm  679 sep  4 11:08 access.log
-rw-r----- 1 root adm  864 sep  4 11:10 error.log
-rw-r----- 1 root adm    0 sep  4 11:07 other_vhosts_access.log
-rw-r--r-- 1 root root  844 sep  4 11:48 sitio2_access.log
-rw-r--r-- 1 root root    0 sep  4 11:10 sitio2_error.log
-rw-r--r-- 1 root root 5085 sep  4 11:56 wwwredes_access.log
-rw-r--r-- 1 root root  486 sep  4 11:16 wwwredes_error.log

```

Listado del directorio /var/log/apache2/ mostrando los archivos globales y los específicos de cada VirtualHost.

```

redes@ubuntu-server:~$ tail -n 20 /var/log/apache2/wwwredes_access.log
192.168.1.31 - - [04/Sep/2026:11:47:49 +0000] "GET /favicon.ico HTTP/1.1" 404 492 "http://wwwredes.local/"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:48:40 +0000] "-" 408 0 "-" "-"
192.168.1.31 - - [04/Sep/2026:11:48:57 +0000] "-" 408 0 "-" "-"
192.168.1.31 - - [04/Sep/2026:11:49:39 +0000] "GET / HTTP/1.1" 200 535 "-" "Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:50:30 +0000] "-" 408 0 "-" "-"
192.168.1.31 - - [04/Sep/2026:11:50:39 +0000] "GET /admin/ HTTP/1.1" 401 745 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:51:30 +0000] "-" 408 0 "-" "-"
192.168.1.31 - - [04/Sep/2026:11:51:30 +0000] "GET /admin/ HTTP/1.1" 401 745 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:51:33 +0000] "GET /admin/ HTTP/1.1" 401 744 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:51:34 +0000] "GET /admin/ HTTP/1.1" 401 744 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:51:55 +0000] "GET /admin HTTP/1.1" 401 745 "-" "Mozilla/5.0 (Windows NT 1
0.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:52:47 +0000] "-" 408 0 "-" "-"
192.168.1.31 - admin [04/Sep/2026:11:56:05 +0000] "GET /admin HTTP/1.1" 301 572 "-" "Mozilla/5.0 (Windows
NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - admin [04/Sep/2026:11:56:05 +0000] "GET /admin/ HTTP/1.1" 200 953 "-" "Mozilla/5.0 (Windows
NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:11:56:56 +0000] "-" 408 0 "-" "-"
192.168.1.31 - - [04/Sep/2026:11:59:56 +0000] "GET /admin/ HTTP/1.1" 401 745 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - redesl [04/Sep/2026:12:00:02 +0000] "GET /admin/ HTTP/1.1" 401 745 "-" "Mozilla/5.0 (Window
s NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:12:00:04 +0000] "GET /admin/ HTTP/1.1" 401 744 "-" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:12:00:04 +0000] "GET /favicon.ico HTTP/1.1" 404 489 "http://192.168.1.2/admi
n/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Saf
ari/537.36"
192.168.1.31 - - [04/Sep/2026:12:00:16 +0000] "GET /noexiste.html HTTP/1.1" 404 490 "-" "Mozilla/5.0 (Wind
ows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"

```

Contenido de wwwredes_access.log.

```

redes@ubuntu-server:~$ sudo tail -n 20 /var/log/apache2/wwwredes_error.log
[Fri Sep 04 11:11:21.353233 2026] [autoindex:error] [pid 3736:tid 13978297937260
8] [client 192.168.1.31:56568] AH01276: Cannot serve directory /var/www/wwwredes
/admin/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,ind
ex.xhtml,index.htm) found, and server-generated directory index forbidden by Opt
ions directive
[Fri Sep 04 11:16:11.366184 2026] [auth_basic:error] [pid 3736:tid 1397828060135
04] [client 192.168.1.31:62050] AH01618: user redes not found: /admin/
[Fri Sep 04 12:00:02.231809 2026] [auth_basic:error] [pid 3736:tid 1397827976208
00] [client 192.168.1.31:56700] AH01618: user redesl not found: /admin/

```

Contenido de wwwredes_error.log.

10. Conclusiones

Se completaron satisfactoriamente la totalidad de los requerimientos planteados en el Ejercicio 1. Se logró establecer el acceso remoto por SSH mediante el cliente PuTTY, desde el cual se administró íntegramente el servidor, y se instaló Apache 2.4.52 sobre una máquina virtual Ubuntu Server 22.04, condición verificada por tres vías independientes.

La configuración de VirtualHosts permitió comprobar en la práctica cómo un único servidor, con una sola dirección IP y un solo puerto de escucha, puede alojar sitios web independientes que se distinguen entre sí por el encabezado Host de la petición HTTP. Se comprendió asimismo la lógica del esquema sites-available / sites-enabled que Ubuntu implementa para administrar dichas configuraciones, y la relevancia del orden alfabético de carga en la determinación del sitio por defecto.

La implementación del control de acceso mediante .htaccess evidenció la dependencia de esta funcionalidad respecto de la directiva AllowOverride, cuyo valor por defecto ignora silenciosamente el archivo, y permitió observar el intercambio completo del esquema de autenticación básica HTTP a través de los códigos 401 y 200 registrados consecutivamente en el log.

Finalmente, el análisis de los registros resultó ser el componente de mayor valor formativo del ejercicio: la lectura de los distintos códigos de estado obtenidos durante las pruebas, junto con su correlato en el registro de errores, permitió comprender el comportamiento interno del servidor ante cada tipo de petición, herramienta indispensable para el diagnóstico de problemas en entornos productivos.

11. Anexo — Comandos utilizados

Se detalla a continuación la secuencia completa de comandos ejecutados durante la resolución del ejercicio, a modo de resumen:

```
# Reconocimiento del entorno
lsb_release -a
ip a
systemd-detect-virt

# Acceso SSH
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo systemctl status ssh
sudo ufw allow OpenSSH
ss -tlnp | grep :22

# Instalacion de Apache
sudo apt install -y apache2
sudo systemctl status apache2
apache2 -v
sudo ufw allow 'Apache Full'
sudo ufw status

# Estructura de directorios y contenido
sudo mkdir -p /var/www/wwwredes/admin
sudo mkdir -p /var/www/sitio2
sudo nano /var/www/wwwredes/index.html
sudo nano /var/www/sitio2/index.html
sudo nano /var/www/wwwredes/admin/index.html
sudo chown -R www-data:www-data /var/www/wwwredes /var/www/sitio2
sudo chmod -R 755 /var/www
sudo chmod 644 /var/www/wwwredes/admin/index.html
```

```
# VirtualHosts
sudo nano /etc/apache2/sites-available/000-wwwredes.conf
sudo nano /etc/apache2/sites-available/sitio2.conf
sudo a2ensite 000-wwwredes.conf
sudo a2ensite sitio2.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
ls -l /etc/apache2/sites-enabled/

# Autenticacion con .htaccess
sudo apt install -y apache2-utils
sudo htpasswd -c /etc/apache2/.htpasswd admin
sudo chown root:www-data /etc/apache2/.htpasswd
sudo chmod 640 /etc/apache2/.htpasswd
sudo nano /var/www/wwwredes/admin/.htaccess
sudo systemctl reload apache2

# Logs
ls -l /var/log/apache2/
sudo tail -n 20 /var/log/apache2/wwwredes_access.log
sudo tail -n 20 /var/log/apache2/wwwredes_error.log
sudo tail -f /var/log/apache2/wwwredes_access.log
grep LogFormat /etc/apache2/apache2.conf
curl -I http://192.168.1.2/noexiste.html
```

12. Ejercicio 2 — Objetivo y alcance

El Ejercicio 2 requiere publicar una nueva página web sobre el mismo servidor ya configurado, pero atendida en el puerto 8080 en lugar del puerto 80. Se solicita realizar las configuraciones necesarias, indicar cuáles son las modificaciones requeridas y adjuntar la captura del navegador del equipo anfitrión accediendo al nuevo sitio.

El ejercicio se resolvió sobre la misma máquina virtual y con la misma instalación de Apache utilizadas en el Ejercicio 1, cuyas características de entorno se detallan en la sección 2 del presente informe. La dirección IP del servidor se mantuvo en 192.168.1.2, lo que se verificó al iniciar la sesión de trabajo, y el servicio Apache continuaba en estado activo desde la configuración previa.

Concepto involucrado: un mismo proceso servidor puede atender simultáneamente varios puertos TCP. Mientras que en el Ejercicio 1 se emplearon VirtualHosts basados en nombre —que comparten el puerto 80 y se diferencian por el encabezado Host de la petición—, en este ejercicio se implementa la diferenciación a nivel de **puerto de escucha**, de modo que el destino de la petición queda determinado por el número de puerto indicado en la URL y no por el nombre del sitio.

13. Puntos 1 y 2 — Configuración del sitio en el puerto 8080

13.1. Análisis previo: el rol de ports.conf

Antes de crear el nuevo host virtual fue necesario comprender un aspecto central de la arquitectura de Apache: la definición de un bloque `<VirtualHost *:8080>` no basta por sí sola para que el servidor atienda ese puerto. La directiva `VirtualHost` únicamente indica a qué combinación de dirección y puerto corresponde una configuración determinada, pero no abre el socket de escucha.

La apertura efectiva del puerto la realiza la directiva `Listen`, declarada de forma global en el archivo `/etc/apache2/ports.conf`, el cual es incluido desde la configuración principal `apache2.conf`. Si se omitiera este paso, Apache arrancaría sin errores pero el puerto 8080 permanecería cerrado, y cualquier intento de conexión sería rechazado por el sistema operativo. Por este motivo la modificación de `ports.conf` constituye el primer paso obligatorio del ejercicio.

13.2. Modificación del archivo ports.conf

Se editó el archivo mediante el editor nano a través de la sesión SSH:

```
sudo nano /etc/apache2/ports.conf
```

Se agregó la directiva `Listen 8080` a continuación de la ya existente para el puerto 80, quedando el archivo con el siguiente contenido relevante:

```
Listen 80
Listen 8080

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

Se conservaron las declaraciones condicionales correspondientes al puerto 443, que solo se aplican si los módulos de cifrado se encuentran habilitados, ya que no forman parte del alcance de este ejercicio.

```
redes@ubuntu-server:~$ cat /etc/apache2/ports.conf
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 80
Listen 8080
<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

Contenido del archivo /etc/apache2/ports.conf obtenido con cat, donde se observen ambas directivas: Listen 80 y Listen 8080.

13.3. Creación del directorio y del contenido HTML

Se creó un directorio independiente para el nuevo sitio, manteniendo el criterio de organización adoptado en el Ejercicio 1 de asignar a cada sitio su propia raíz documental:

```
sudo mkdir -p /var/www/sitio8080
```

A continuación se generó el archivo index.html con el contenido exacto provisto por la cátedra:

```
sudo tee /var/www/sitio8080/index.html > /dev/null <<'EOF'
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1">
    <!-- Title Page -->
    <title> Mi Sitio WEB - Puerto 8080</title>
</head>
<body>
    <div id="container">
<H1> Nuevo sitio de Redes de Datos - Acceso a traves del puerto 8080</H1>
    </div>
</body>
</html>
EOF
```

Se utilizó tee con un here-document delimitado por comillas simples (<<'EOF') para que el intérprete de comandos no realice sustitución de variables ni interprete los caracteres especiales del código HTML, garantizando que el archivo se escriba de manera literal.

Finalmente se asignaron el propietario y los permisos correspondientes, siguiendo el mismo criterio fundamentado en la sección 5.1:

```
sudo chown -R www-data:www-data /var/www/sitio8080
sudo chmod -R 755 /var/www/sitio8080
```

13.4. Creación del VirtualHost

Se creó el archivo de configuración /etc/apache2/sites-available/sitio8080.conf:

```
sudo nano /etc/apache2/sites-available/sitio8080.conf
```

```
<VirtualHost *:8080>
    ServerName sitio8080.local
    ServerAdmin admin@sitio8080.local
    DocumentRoot /var/www/sitio8080

    <Directory /var/www/sitio8080>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/sitio8080_error.log
    CustomLog ${APACHE_LOG_DIR}/sitio8080_access.log combined
</VirtualHost>
```

La diferencia sustancial respecto de los VirtualHosts del Ejercicio 1 reside en el encabezado del bloque: mientras aquellos declaraban `<VirtualHost *:80>`, este declara `<VirtualHost *:8080>`. El asterisco indica que la configuración se aplica a peticiones recibidas por cualquiera de las direcciones IP del servidor, y el número que sigue a los dos puntos determina el puerto al cual queda asociado el sitio.

El resto de las directivas conserva el mismo significado ya documentado en la sección 5.3. Se mantuvieron los archivos de registro propios para este sitio, de modo que su tráfico quede aislado del correspondiente a los sitios del puerto 80.

```
redes@ubuntu-server:~$ cat /etc/apache2/sites-available/sitio8080.conf
<VirtualHost *:8080>
    ServerName sitio8080.local
    ServerAdmin admin@sitio8080.local
    DocumentRoot /var/www/sitio8080

    <Directory /var/www/sitio8080>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/sitio8080_error.log
    CustomLog ${APACHE_LOG_DIR}/sitio8080_access.log combined
</VirtualHost>
```

Contenido del archivo /etc/apache2/sites-available/sitio8080.conf obtenido con cat, mostrando el bloque VirtualHost completo con el puerto 8080.

13.5. Habilitación del sitio y validación de la configuración

Se habilitó el nuevo sitio mediante la herramienta `a2ensite`, que genera el enlace simbólico correspondiente en el directorio `sites-enabled`, se validó la sintaxis y se recargó el servicio:

```
sudo a2ensite sitio8080.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

La validación devolvió `Syntax OK`, confirmando que tanto la nueva directiva `Listen` como el bloque `VirtualHost` eran sintácticamente correctos. Se optó nuevamente por `reload` en lugar de `restart` para aplicar los cambios sin interrumpir las conexiones activas ni los sitios ya publicados en el puerto 80.

```

redes@ubuntu-server:~$ sudo chown -R www-data:www-data /var/www/sitio8080
sudo chmod -R 755 /var/www/sitio8080
redes@ubuntu-server:~$ sudo nano /etc/apache2/sites-available/sitio8080.conf
redes@ubuntu-server:~$ sudo a2ensite sitio8080.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
Enabling site sitio8080.
To activate the new configuration, you need to run:
    systemctl reload apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK

```

Terminal mostrando la salida de `a2ensite sitio8080.conf` y el resultado "Syntax OK" de `apache2ctl configtest`.

13.6. Regla de firewall

Se incorporó la regla correspondiente al nuevo puerto:

```
sudo ufw allow 8080/tcp
```

La ejecución devolvió `Rules updated` y `Rules updated (v6)`, indicando que la regla se registró tanto para IPv4 como para IPv6. Se reitera lo señalado en la sección 4.3: UFW se encuentra inactivo en esta instalación, por lo que la regla queda precargada y solo tendría efecto al habilitar el firewall. A diferencia de Apache, que dispone de un perfil de aplicación predefinido para los puertos 80 y 443, el puerto 8080 no corresponde a ningún perfil registrado, motivo por el cual debió especificarse explícitamente el número de puerto y el protocolo.

14. Punto 3 — Modificaciones necesarias y verificación

14.1. Síntesis de las modificaciones realizadas

En respuesta a lo solicitado por el enunciado, se detallan a continuación las modificaciones que resultaron necesarias para publicar el sitio en el puerto 8080:

Modificación	Archivo o recurso	Necesidad
Declaración del puerto de escucha	<code>/etc/apache2/ports.conf</code>	Agregar <code>Listen 8080</code> . Es la única directiva que abre efectivamente el socket TCP. Sin ella el puerto permanece cerrado.
Creación de la raíz documental	<code>/var/www/sitio8080</code>	Directorio independiente donde reside el contenido del nuevo sitio.
Publicación del contenido	<code>/var/www/sitio8080/index.html</code>	Archivo HTML provisto por la cátedra, servido como índice del sitio.
Asignación de propiedad y permisos	<code>www-data:www-data</code> , modo <code>755</code>	Permite que los procesos hijos de Apache lean y atraviesen el directorio.
Definición del host virtual	<code>/etc/apache2/sites-available/sitio8080.conf</code>	Asocia el puerto 8080 con la raíz documental y define sus permisos y registros propios.
Activación del sitio	Enlace simbólico en <code>sites-enabled/</code>	Generado con <code>a2ensite</code> . Sin el enlace, la configuración existe pero no se carga.

Modificación	Archivo o recurso	Necesidad
Aplicación de la configuración	systemctl reload apache2	Relee la configuración y abre el nuevo puerto sin interrumpir el servicio.
Regla de firewall	ufw allow 8080/tcp	Habilita el tráfico entrante al nuevo puerto en caso de activarse UFW.

14.2. Verificación de los puertos en escucha

Se comprobó que el servidor estuviera efectivamente escuchando en ambos puertos mediante la herramienta ss, que muestra los sockets abiertos junto con el proceso propietario:

```
sudo ss -tlnp | grep -E ':\80|:\8080'
```

La salida obtenida fue la siguiente:

```
LISTEN 0 511 *:80 *: users:(("apache2",pid=1235,fd=4),
      ("apache2",pid=1234,fd=4),
      ("apache2",pid=708,fd=4))
LISTEN 0 511 *:8080 *: users:(("apache2",pid=1235,fd=6),
      ("apache2",pid=1234,fd=6),
      ("apache2",pid=708,fd=6))
```

Interpretación de la salida:

- Se observan dos sockets en estado LISTEN, uno por cada puerto configurado (80 y 8080), ambos asociados a la dirección comodín *, lo que significa que aceptan conexiones por cualquier interfaz del servidor.
- Los identificadores de proceso son **los mismos en ambas líneas** (708, 1234 y 1235). Esto evidencia que no se trata de dos servidores independientes, sino de un único servicio Apache —con su proceso padre y sus procesos hijos— atendiendo simultáneamente los dos puertos. Es precisamente el comportamiento que el ejercicio busca demostrar.
- Los descriptores de archivo difieren entre puertos (fd=4 para el 80 y fd=6 para el 8080), confirmando que cada puerto corresponde a un socket distinto dentro del mismo proceso.
- El valor 511 indica el tamaño de la cola de conexiones pendientes (backlog) configurado para cada socket.

```
redes@ubuntu-server:~$ sudo ss -tlnp
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port
Process
LISTEN     0         4096      127.0.0.53%lo:53      0.0.0.0:*
  users:(("systemd-resolve",pid=618,fd=14))
LISTEN     0         128       0.0.0.0:22           0.0.0.0:*
  users:(("sshd",pid=696,fd=3))
LISTEN     0         128       [::]:22             [::]:*
  users:(("sshd",pid=696,fd=4))
LISTEN     0         511       *:80                 *:~
  users:(("apache2",pid=1235,fd=4),("apache2",pid=1234,fd=4),("apache2",pid=708,fd=4))
LISTEN     0         511       *:8080                *:~
  users:(("apache2",pid=1235,fd=6),("apache2",pid=1234,fd=6),("apache2",pid=708,fd=6))
```

Salida del comando sudo ss -tlnp mostrando Apache en escucha simultánea sobre los puertos 80 y 8080 con los mismos PIDs.

14.3. Verificación mediante curl

Desde la propia máquina virtual se realizó una petición HTTP solicitando únicamente los encabezados de respuesta:

```
curl -I http://localhost:8080
```

La respuesta obtenida fue:

```
HTTP/1.1 200 OK
Date: Mon, 07 Sep 2026 20:19:38 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Mon, 07 Sep 2026 20:03:20 GMT
ETag: "143-65aea1c1ae614"
Accept-Ranges: bytes
Content-Length: 323
Vary: Accept-Encoding
Content-Type: text/html
```

Análisis de los encabezados recibidos:

Encabezado	Significado
HTTP/1.1 200 OK	Línea de estado. El servidor procesó la petición correctamente y el recurso existe.
Date	Momento en que el servidor generó la respuesta, expresado en GMT.
Server	Software y versión del servidor: Apache/2.4.52 (Ubuntu), la misma instancia que atiende el puerto 80.
Last-Modified	Fecha de última modificación del archivo, correspondiente al momento de su creación.
ETag	Identificador de versión del recurso, utilizado para la validación de caché.
Accept-Ranges	Indica que el servidor admite peticiones parciales del recurso.
Content-Length	Tamaño del cuerpo de la respuesta: 323 bytes, correspondientes al archivo index.html.
Content-Type	Tipo de contenido: text/html, determinado por la extensión del archivo.

Observación adicional: la primera porción del ETag generado por Apache corresponde al tamaño del archivo expresado en notación hexadecimal. En este caso el valor 143 en hexadecimal equivale a 323 en decimal, coincidiendo exactamente con el encabezado Content-Length. Este detalle permite verificar la consistencia interna de la respuesta.

Cabe aclarar que esta prueba se realizó contra localhost, es decir desde el propio servidor, por lo que valida el funcionamiento del servicio pero no la accesibilidad desde la red. Dicha verificación se documenta en el apartado siguiente.

```
redes@ubuntu-server:~$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Date: Mon, 07 Sep 2026 20:19:38 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Mon, 07 Sep 2026 20:03:20 GMT
ETag: "143-65aealc1ae614"
Accept-Ranges: bytes
Content-Length: 323
Vary: Accept-Encoding
Content-Type: text/html
```

Salida del comando `curl -I http://localhost:8080` mostrando la respuesta HTTP/1.1 200 OK con todos sus encabezados.

14.4. Verificación desde el navegador del equipo anfitrión

Tal como solicita expresamente el enunciado, se accedió desde el navegador del equipo anfitrión a la dirección `http://192.168.1.2:8080`, obteniéndose correctamente la página solicitada con el encabezado "Nuevo sitio de Redes de Datos - Acceso a través del puerto 8080".

Se destaca que, a diferencia del acceso al puerto 80, en este caso resulta obligatorio especificar el número de puerto en la URL. Ello se debe a que el puerto 80 es el valor por defecto del esquema `http://` y los navegadores lo asumen implícitamente; cualquier otro puerto debe indicarse de manera explícita mediante la sintaxis `host:puerto`.

Complementariamente se accedió a la dirección del servidor sin especificar puerto, comprobándose que continúa respondiendo el sitio configurado en el Ejercicio 1. De este modo queda demostrado que ambos sitios coexisten de forma independiente sobre la misma máquina y la misma dirección IP, diferenciándose únicamente por el puerto de escucha.



Navegador del equipo anfitrión accediendo a `http://192.168.1.2:8080`, con la URL visible en la barra de direcciones y el título "Mi Sitio WEB - Puerto 8080" en la pestaña.



Comparativa en dos pestañas del navegador: <http://192.168.1.2> (sitio del Ejercicio 1) y <http://192.168.1.2:8080> (sitio del Ejercicio 2), evidenciando la coexistencia de ambos.

15. Registros generados por el nuevo sitio

Como consecuencia de las directivas ErrorLog y CustomLog declaradas dentro del VirtualHost, Apache generó archivos de registro exclusivos para el sitio del puerto 8080, independientes de los correspondientes a los sitios del puerto 80:

```
ls -l /var/log/apache2/  
sudo tail -n 10 /var/log/apache2/sitio8080_access.log
```

Las entradas registradas siguen el formato combined ya analizado en la sección 9.3. En ellas puede observarse la dirección IP del equipo anfitrión como origen de las peticiones realizadas desde el navegador, el método GET, el recurso solicitado y el código de estado 200 correspondiente a las entregas exitosas del archivo index.html.

La separación de registros por sitio resulta especialmente útil en este escenario: permite distinguir con claridad qué tráfico corresponde a cada puerto sin necesidad de filtrar un archivo común, facilitando tanto el monitoreo como el diagnóstico de eventuales fallas en cada servicio de manera independiente.

```
redes@ubuntu-server:~$ sudo tail -n 10 /var/log/apache2/sitio8080_access.log  
127.0.0.1 - - [07/Sep/2026:20:04:22 +0000] "HEAD / HTTP/1.1" 200 252 "-" "curl/7.81.0"  
192.168.1.31 - - [07/Sep/2026:20:04:38 +0000] "GET / HTTP/1.1" 200 583 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36 Edg/152.0.0.0"  
192.168.1.31 - - [07/Sep/2026:20:04:38 +0000] "GET /favicon.ico HTTP/1.1" 404 491 "http://192.168.1.2:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36 Edg/152.0.0.0"  
127.0.0.1 - - [07/Sep/2026:20:19:38 +0000] "HEAD / HTTP/1.1" 200 252 "-" "curl/7.81.0"  
192.168.1.31 - - [07/Sep/2026:20:20:48 +0000] "GET / HTTP/1.1" 200 583 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"  
192.168.1.31 - - [07/Sep/2026:20:20:48 +0000] "GET /favicon.ico HTTP/1.1" 404 491 "http://192.168.1.2:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"  
192.168.1.31 - - [07/Sep/2026:20:21:39 +0000] "-" 408 0 "-" "-"  
192.168.1.31 - - [07/Sep/2026:20:22:30 +0000] "-" 408 0 "-" "-"
```

Contenido del archivo `sitio8080_access.log` mostrando las peticiones al nuevo sitio con sus códigos de estado.

16. Conclusiones del Ejercicio 2

Se cumplieron satisfactoriamente los requerimientos planteados. El nuevo sitio quedó publicado y accesible en el puerto 8080 desde el navegador del equipo anfitrión, coexistiendo con los sitios previamente configurados en el puerto 80 sobre la misma instancia de Apache.

El aprendizaje central del ejercicio radica en la distinción entre dos directivas que suelen confundirse: `Listen`, de alcance global, es la única que abre efectivamente un socket de escucha en el sistema operativo, mientras que `VirtualHost` únicamente asocia una configuración a una combinación de dirección y puerto previamente habilitada. Omitir la primera conduce a un escenario particularmente engañoso, en el cual el servidor arranca sin reportar error alguno y la validación de sintaxis resulta exitosa, pero el puerto permanece inaccesible.

La verificación mediante la herramienta `ss` resultó determinante para comprobar el comportamiento esperado: la coincidencia de los identificadores de proceso entre ambos puertos demuestra que un único servicio Apache atiende simultáneamente múltiples sockets, en contraposición a la alternativa de ejecutar instancias separadas del servidor.

Finalmente, el ejercicio permitió contrastar los dos mecanismos de multiplexación de sitios que ofrece Apache: la diferenciación por nombre, empleada en el Ejercicio 1, donde varios sitios comparten un mismo puerto y se distinguen por el encabezado `Host` de la petición; y la diferenciación por puerto, aplicada aquí, donde el destino queda determinado por el número de puerto especificado en la URL.

17. Anexo — Comandos utilizados en el Ejercicio 2

```
# Verificacion del entorno al reiniciar la sesion
ip a | grep inet
sudo systemctl status apache2

# Declaracion del puerto de escucha
sudo nano /etc/apache2/ports.conf
cat /etc/apache2/ports.conf

# Creacion del sitio
sudo mkdir -p /var/www/sitio8080
sudo nano /var/www/sitio8080/index.html
sudo chown -R www-data:www-data /var/www/sitio8080
sudo chmod -R 755 /var/www/sitio8080

# VirtualHost
sudo nano /etc/apache2/sites-available/sitio8080.conf
sudo a2ensite sitio8080.conf
sudo apache2ctl configtest
sudo systemctl reload apache2

# Firewall
sudo ufw allow 8080/tcp

# Verificaciones
sudo ss -tlnp | grep -E ':80|:8080'
curl -I http://localhost:8080
ls -l /var/log/apache2/
sudo tail -n 10 /var/log/apache2/sitio8080_access.log
```

18. Ejercicio 3 — Objetivo y alcance

El Ejercicio 3 propone analizar el protocolo HTTP a nivel de mensaje, empleando la herramienta curl desde el equipo anfitrión para observar de manera directa los encabezados y el cuerpo de las respuestas emitidas por el servidor. Se solicita identificar y explicar los campos principales de la respuesta, contrastar dicha información con la que muestra un navegador web, analizar el comportamiento del servidor ante la solicitud de un recurso inexistente y, finalmente, examinar el registro de accesos identificando los componentes de una entrada.

El ejercicio se realizó sobre el mismo entorno documentado en la sección 2, con el servidor Apache respondiendo en la dirección 192.168.1.2 y el equipo anfitrión Windows como cliente, con dirección 192.168.1.31.

19. Punto 1 — Análisis de encabezados HTTP mediante curl

19.1. Consideración previa sobre la ejecución en Windows

Al ejecutar el comando curl desde el Símbolo del sistema de Windows, el intérprete devolvió el mensaje "curl no se reconoce como un comando interno o externo". Windows 11 incorpora el binario de curl dentro del sistema, pero en esta instalación su directorio no se encontraba incluido en la variable de entorno PATH. Se resolvió invocando el ejecutable mediante su ruta absoluta:

```
C:\Windows\System32\curl.exe -i http://192.168.1.2
```

Se deja constancia adicional de que, en caso de utilizar PowerShell en lugar del Símbolo del sistema, el nombre curl se encuentra definido como alias del cmdlet Invoke-WebRequest, el cual no admite las opciones -i ni -I. Por ese motivo las pruebas se realizaron desde el Símbolo del sistema invocando explícitamente el ejecutable.

19.2. Respuesta obtenida con curl -i

La opción -i (include) indica a curl que muestre los encabezados de la respuesta junto con el cuerpo del mensaje. La salida obtenida fue la siguiente:

```
HTTP/1.1 200 OK
Date: Mon, 07 Sep 2026 20:30:55 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Fri, 04 Sep 2026 11:09:45 GMT
ETag: "fb-65aa64e4e3690"
Accept-Ranges: bytes
Content-Length: 251
Vary: Accept-Encoding
Content-Type: text/html

<!doctype html>
<html lang="es">
<head><meta charset="utf-8"><title>Redes de Datos - Sitio principal</title></head>
<body>
  <h1>Servidor Apache - Redes de Datos 2026</h1>
  <p>Sitio publicado en el puerto 80 - UNNE - Tobias Orban</p>
</body>
</html>
```

Se observa con claridad la estructura de un mensaje de respuesta HTTP: una línea de estado, un bloque de encabezados, una línea en blanco que actúa como separador obligatorio, y finalmente el cuerpo del mensaje. Esta separación mediante línea vacía es la que permite al cliente determinar dónde terminan los metadatos y dónde comienza el contenido.

```

C:\Users\Usuario>C:\Windows\System32\curl.exe -i http://192.168.1.2
HTTP/1.1 200 OK
Date: Mon, 07 Sep 2026 20:30:55 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Fri, 04 Sep 2026 11:09:45 GMT
ETag: "fb-65aa64e4e3690"
Accept-Ranges: bytes
Content-Length: 251
Vary: Accept-Encoding
Content-Type: text/html

<!doctype html>
<html lang="es">
<head><meta charset="utf-8"><title>Redes de Datos - Sitio principal</title></head>
<body>
  <h1>Servidor Apache - Redes de Datos 2026</h1>
  <p>Sitio publicado en el puerto 80 - UNNE - Tobias Orban</p>
</body>
</html>

C:\Users\Usuario>

```

Salida completa del comando curl.exe -i http://192.168.1.2 en el Símbolo del sistema, mostrando encabezados y cuerpo HTML.

19.3. Identificación y explicación de los campos solicitados

Campo	Valor obtenido	Explicación
Status Line	HTTP/1.1 200 OK	Primera línea de toda respuesta HTTP. Se compone de la versión del protocolo, el código numérico de estado y su frase de razón. El código 200 pertenece a la familia 2xx e indica que la petición fue recibida, comprendida y procesada correctamente.
Date	Mon, 07 Sep 2026 20:30:55 GMT	Momento en que el servidor generó la respuesta. Se expresa obligatoriamente en GMT según el RFC 7231, con independencia de la zona horaria del servidor, para garantizar una referencia temporal común entre sistemas.
Server	Apache/2.4.52 (Ubuntu)	Identifica el software y la versión que atendió la petición. Constituye información sensible desde el punto de vista de la seguridad, ya que facilita la búsqueda de vulnerabilidades conocidas para esa versión; en entornos productivos se restringe mediante la directiva ServerTokens.
Content-Length	251	Tamaño en bytes del cuerpo del mensaje, sin incluir los encabezados. Permite al cliente conocer de antemano cuántos bytes debe recibir y determinar con precisión cuándo la transferencia ha concluido.
Content-Type	text/html	Tipo MIME del contenido. Indica al cliente cómo debe interpretar la secuencia de bytes recibida; en este caso, como un documento HTML a renderizar y no como texto plano o un archivo a descargar.

19.4. Encabezados adicionales observados

La respuesta incluyó otros encabezados que, si bien no fueron solicitados expresamente, resultan relevantes para comprender el funcionamiento del protocolo:

- **Last-Modified:** informa la fecha de última modificación del recurso (04/09/2026), tres días anterior al valor de Date (07/09/2026). Esta diferencia evidencia la distinción entre el momento en que se generó la respuesta y el momento en que se modificó el archivo por última vez, correspondiente a su creación durante el Ejercicio 1.
- **ETag:** identificador de versión del recurso, empleado por los mecanismos de caché para validar si una copia almacenada sigue vigente. Resulta destacable que su primer componente, fb, corresponde al tamaño del archivo expresado en hexadecimal: 0xFB equivale a 251 en decimal, coincidiendo exactamente con el encabezado Content-Length.
- **Accept-Ranges: bytes:** indica que el servidor admite peticiones de rangos parciales del recurso, funcionalidad que habilita la reanudación de descargas interrumpidas.
- **Vary: Accept-Encoding:** advierte a los intermediarios de caché que la respuesta puede diferir según el valor del encabezado Accept-Encoding enviado por el cliente, evitando que se entregue una versión comprimida a un cliente que no admite compresión.

19.5. Diferencia respecto de la información mostrada por el navegador

El enunciado solicita explicitar qué diferencia existe entre la información que muestra curl y la que presenta un navegador web. La respuesta central es que ambos clientes reciben exactamente el mismo mensaje HTTP: la diferencia no reside en lo que el servidor envía, sino en lo que cada cliente decide mostrar al usuario.

El navegador consume los encabezados de manera interna —los utiliza para determinar la codificación, gestionar la caché y decidir cómo renderizar el contenido— y presenta en pantalla únicamente el resultado visual del documento HTML. Curl, en cambio, no interpreta ni renderiza: expone el intercambio en su forma original.

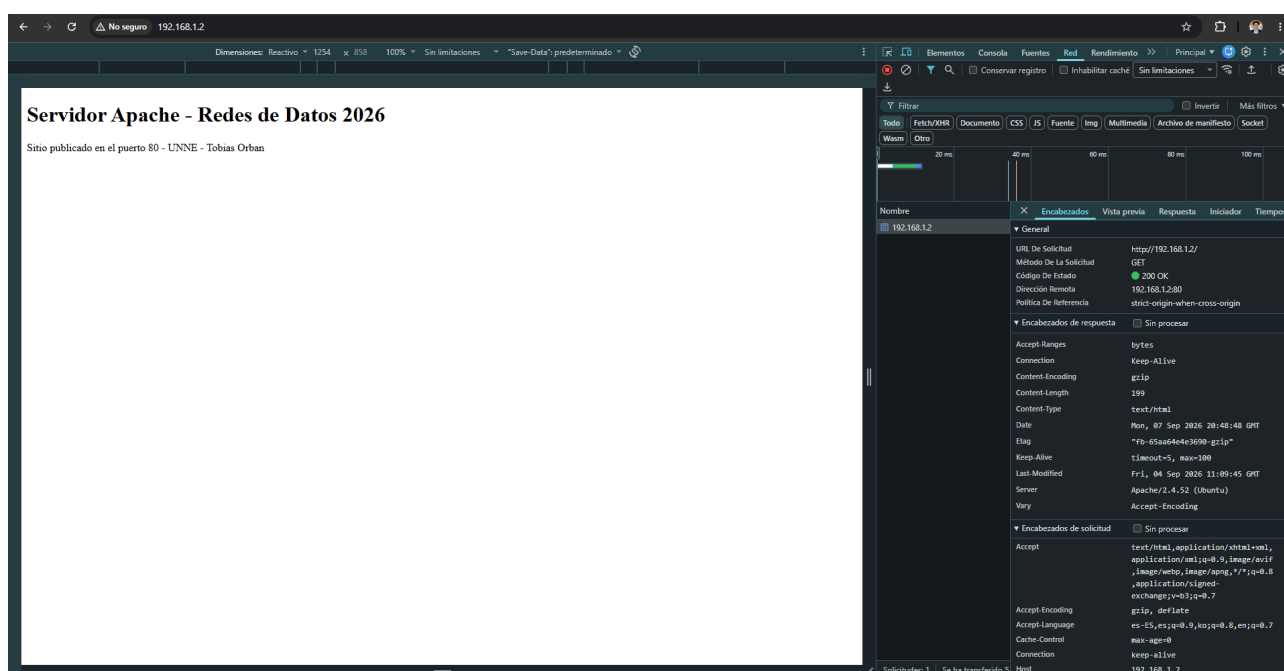
Para comprobarlo empíricamente se accedió a la misma URL desde el navegador y se abrieron las herramientas de desarrollador (F12), en la pestaña Red. Los datos observados fueron los siguientes:

Dato mostrado por el navegador	Valor	Correlación con curl
Solicitudes	1	El navegador realizó una única petición, la misma que curl.
Estado	200	Coincide con la línea de estado devuelta por curl.
Tipo	document	Se corresponde con el Content-Type: text/html.
Recursos	251 B	Coincide exactamente con el Content-Length informado por curl.
Transferido	534 B	Supera al cuerpo en aproximadamente 283 bytes, correspondientes a los encabezados de la respuesta.
DOMContentLoaded	41 ms	Métrica de renderizado propia del navegador, sin equivalente en curl.

La diferencia entre los valores **Recursos (251 B)** y **Transferido (534 B)** constituye la demostración más clara del concepto: el navegador efectivamente recibió los encabezados —esos bytes adicionales— aunque no los exhiba en la ventana principal. Para visualizarlos es necesario recurrir a las herramientas de desarrollador, en la sección Response Headers.

Diferencias adicionales entre ambos clientes:

- El navegador genera **peticiones automáticas adicionales** que curl no realiza, como la solicitud del archivo `favicon.ico`, cuya ausencia quedó registrada como código 404 en el log del servidor.
- El navegador aplica políticas de caché, sigue redirecciones automáticamente y ejecuta el código JavaScript embebido; curl no realiza ninguna de estas acciones salvo indicación expresa.
- Curl permite observar el mensaje HTTP en su forma original, lo que lo convierte en la herramienta adecuada para el diagnóstico del protocolo, mientras que el navegador está orientado a la presentación del contenido al usuario final.



Herramientas de desarrollador del navegador (F12), pestaña Red, mostrando la petición a 192.168.1.2

19.6. Respuesta ante un recurso inexistente

Se ejecutó a continuación el comando solicitado por el enunciado, dirigido a un recurso que no existe en el servidor:

```
C:\Windows\System32\curl.exe -I http://192.168.1.2/noexiste.html
```

La respuesta obtenida fue:

```
HTTP/1.1 404 Not Found
Date: Mon, 07 Sep 2026 20:36:29 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Type: text/html; charset=iso-8859-1
```

La explicación de esta respuesta comprende dos aspectos diferenciados:

1) El código de estado. El servidor devolvió 404 Not Found, perteneciente a la familia 4xx (errores del cliente). Indica que la petición fue sintácticamente correcta y el servidor la comprendió, pero el recurso

solicitado no existe en la ruta indicada. Corresponde señalar que el servidor continúa funcionando correctamente: el error no es del servicio sino de la solicitud.

2) La ausencia de cuerpo. La opción `-I` en mayúscula no equivale a `-i`: mientras esta última solicita el recurso mediante el método GET y muestra encabezados y cuerpo, `-I` emite una petición con el método HEAD. Dicho método está definido en el protocolo para obtener únicamente los metadatos de un recurso, sin transferir su contenido. Por ese motivo la respuesta carece de cuerpo y tampoco incluye el encabezado Content-Length.

Para verificar esta interpretación se repitió la misma petición empleando la opción en minúscula:

```
C:\Windows\System32\curl.exe -i http://192.168.1.2/noexiste.html

HTTP/1.1 404 Not Found
Date: Mon, 07 Sep 2026 20:36:52 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Length: 273
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 192.168.1.2 Port 80</address>
</body></html>
```

La comparación entre ambas ejecuciones permite extraer las siguientes conclusiones:

- El encabezado Content-Type aparecía ya en la respuesta a HEAD, anticipando que el servidor disponía de un documento HTML para entregar. La petición con GET confirma su existencia: se trata de la página de error generada internamente por Apache, de 273 bytes.
- La declaración `charset=iso-8859-1` corresponde a la codificación por defecto que Apache aplica a sus páginas de error autogeneradas, y no a la del sitio publicado.
- La página de error expone en su última línea la versión del servidor, la dirección IP y el puerto: `Apache/2.4.52 (Ubuntu) Server at 192.168.1.2 Port 80`. Esta divulgación de información resulta indeseable en entornos productivos y puede suprimirse mediante las directivas `ServerSignature Off` y `ServerTokens Prod`.
- El método HEAD resulta de utilidad práctica para verificar la existencia de un recurso, su tamaño o su fecha de modificación sin incurrir en el costo de transferir su contenido, lo que lo hace especialmente adecuado para tareas de monitoreo automatizado.

```

C:\Users\Usuario>C:\Windows\System32\curl.exe -I http://192.168.1.2/noexiste.html
HTTP/1.1 404 Not Found
Date: Mon, 07 Sep 2026 20:36:29 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Type: text/html; charset=iso-8859-1

C:\Users\Usuario>C:\Windows\System32\curl.exe -i http://192.168.1.2/noexiste.html
HTTP/1.1 404 Not Found
Date: Mon, 07 Sep 2026 20:36:52 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Length: 273
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 192.168.1.2 Port 80</address>
</body></html>

```

Ejecución consecutiva de curl -I y curl -i sobre http://192.168.1.2/noexiste.html, evidenciando el mismo código 404 con y sin cuerpo de respuesta.

20. Punto 2 — Visualización y análisis de los logs

20.1. Consulta del registro

Desde la sesión SSH establecida mediante PuTTY se consultaron las últimas entradas del registro de accesos del sitio principal:

```
sudo tail -n 20 /var/log/apache2/wwwredes_access.log
```

En la salida se identificaron con claridad las peticiones generadas durante este ejercicio, distinguibles por el campo User-Agent, que en lugar de la cadena de identificación de un navegador presenta el valor correspondiente a la herramienta utilizada:

```

192.168.1.31 - - [07/Sep/2026:20:30:55 +0000] "GET / HTTP/1.1"
200 502 "-" "curl/8.21.0"

192.168.1.31 - - [07/Sep/2026:20:36:29 +0000] "HEAD /noexiste.html HTTP/1.1"
404 140 "-" "curl/8.21.0"

192.168.1.31 - - [07/Sep/2026:20:36:52 +0000] "GET /noexiste.html HTTP/1.1"
404 434 "-" "curl/8.21.0"

```

```

redes@ubuntu-server:~$ sudo tail -n 20 /var/log/apache2/wwwredes_access.log
192.168.1.31 - redes1 [04/Sep/2026:12:00:02 +0000] "GET /admin/ HTTP/1.1" 401 745 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:12:00:04 +0000] "GET /admin/ HTTP/1.1" 401 744 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:12:00:04 +0000] "GET /favicon.ico HTTP/1.1" 404 489 "http://192.168.1.2/admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [04/Sep/2026:12:00:16 +0000] "GET /noexiste.html HTTP/1.1" 404 490 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:21:28 +0000] "GET / HTTP/1.1" 200 535 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:21:28 +0000] "GET /favicon.ico HTTP/1.1" 404 489 "http://192.168.1.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:22:20 +0000] "-" 408 0 "-" "-"
192.168.1.31 - - [07/Sep/2026:20:30:55 +0000] "GET / HTTP/1.1" 200 502 "-" "curl/8.21.0"
192.168.1.31 - - [07/Sep/2026:20:36:29 +0000] "HEAD /noexiste.html HTTP/1.1" 404 140 "-" "curl/8.21.0"
192.168.1.31 - - [07/Sep/2026:20:36:52 +0000] "GET /noexiste.html HTTP/1.1" 404 434 "-" "curl/8.21.0"
192.168.1.31 - - [07/Sep/2026:20:37:34 +0000] "GET / HTTP/1.1" 200 535 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:37:57 +0000] "GET / HTTP/1.1" 200 535 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:37:57 +0000] "GET / HTTP/1.1" 200 534 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:38:26 +0000] "GET / HTTP/1.1" 200 535 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:38:26 +0000] "GET / HTTP/1.1" 200 534 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:38:27 +0000] "GET / HTTP/1.1" 200 534 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:38:29 +0000] "GET / HTTP/1.1" 200 534 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:38:29 +0000] "GET /favicon.ico HTTP/1.1" 404 489 "http://192.168.1.2/" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"
192.168.1.31 - - [07/Sep/2026:20:38:33 +0000] "GET / HTTP/1.1" 200 534 "-" "Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36"

```

Salida de `sudo tail -n 20 /var/log/apache2/wwwredes_access.log` mostrando las tres peticiones realizadas con curl junto al tráfico previo del navegador.

20.2. Análisis de una entrada del registro

Se selecciona la primera de las entradas citadas, correspondiente a la petición realizada con curl -i sobre la raíz del sitio:

```
192.168.1.31 - - [07/Sep/2026:20:30:55 +0000] "GET / HTTP/1.1" 200 502 "-" "curl/8.21.0"
```

Identificación de los elementos solicitados por el enunciado:

Elemento solicitado	Valor	Observación
IP cliente	192.168.1.31	Dirección del equipo anfitrión Windows desde el cual se ejecutó el comando.
Método HTTP	GET	Método de solicitud empleado por curl con la opción -i.
Recurso solicitado	/	Raíz del sitio, que el servidor resuelve entregando el archivo index.html del DocumentRoot.
Código HTTP	200	Respuesta satisfactoria: el recurso fue localizado y entregado correctamente.

20.3. Observaciones derivadas del análisis

Registro diferenciado del método HEAD. La segunda entrada registra explícitamente "HEAD /noexiste.html HTTP/1.1", distinguiéndose de la tercera, que consigna "GET /noexiste.html HTTP/1.1". El registro preserva el método exacto de cada solicitud, lo que confirma en el servidor la interpretación realizada en la sección 19.6 sobre la diferencia entre las opciones -l e -i.

Interpretación del campo de bytes. Los valores registrados presentan una particularidad que merece análisis. La petición GET a la raíz consignó 502 bytes, cuando el Content-Length informado por curl fue de 251. De modo análogo, la petición HEAD al recurso inexistente registró 140 bytes pese a no haber transferido cuerpo alguno.

La explicación radica en que la definición del formato combined que incorpora Debian y Ubuntu no emplea la directiva %b —bytes del cuerpo— sino %O, que contabiliza el total de bytes enviados al cliente, encabezados incluidos. Ello explica de manera consistente los tres valores observados: en la petición HEAD los 140 bytes corresponden exclusivamente a los encabezados, ya que no hubo cuerpo; en la petición GET al mismo recurso, los 434 bytes resultan de sumar a esos encabezados el cuerpo de 273 bytes de la página de error junto con el encabezado Content-Length adicional; y en la petición a la raíz, los 502 bytes comprenden el documento de 251 bytes más un bloque de encabezados de magnitud equivalente.

Esta observación puede verificarse consultando la definición del formato en el archivo de configuración principal:

```
grep LogFormat /etc/apache2/apache2.conf
```

```
redes@ubuntu-server:~$ grep LogFormat /etc/apache2/apache2.conf
LogFormat "%v:%p %h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\"" vhost_combined
LogFormat "%h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\"" combined
LogFormat "%h %l %u %t \"%r\" %>s %O" common
LogFormat "%{Referer}i -> %U" referer
LogFormat "%{User-agent}i" agent
```

Salida del comando `grep LogFormat /etc/apache2/apache2.conf`, donde se observa que la definición del formato combined utiliza la directiva %O.

Trazabilidad del origen de las peticiones. Todas las entradas generadas en este ejercicio registran como IP cliente la dirección 192.168.1.31, correspondiente al equipo anfitrión, en contraste con la prueba realizada durante el Ejercicio 1 mediante curl ejecutado desde la propia máquina virtual, que había quedado asentada con la dirección 192.168.1.2. El registro refleja con precisión el origen real de cada solicitud.

Identificación del cliente. El campo User-Agent permite distinguir de manera inequívoca el software que originó cada petición. Junto a las entradas de curl/8.21.0 coexisten registros de Chrome/152.0.0.0 sobre Windows, generados durante las verificaciones con el navegador. Este campo constituye una herramienta habitual para el análisis de tráfico, la detección de clientes automatizados y la elaboración de estadísticas de uso.

Peticiones automáticas del navegador. Las entradas correspondientes a GET /favicon.ico con código 404 aparecen únicamente asociadas a los User-Agent de navegadores, nunca a curl. Ello constituye evidencia empírica de una de las diferencias señaladas en la sección 19.5: el navegador solicita recursos complementarios de forma automática, mientras que curl se limita estrictamente al recurso indicado.

21. Conclusiones del Ejercicio 3

El ejercicio permitió observar el protocolo HTTP en su forma original, sin la mediación interpretativa del navegador. La estructura del mensaje de respuesta —línea de estado, encabezados, línea en blanco y cuerpo— resultó directamente verificable, y cada uno de los campos analizados pudo asociarse a una función concreta dentro del intercambio entre cliente y servidor.

La comparación entre curl y el navegador esclareció un punto conceptualmente relevante: ambos clientes reciben exactamente el mismo mensaje del servidor, y la diferencia se sitúa en el tratamiento que cada uno da a esa información. La discrepancia entre los 251 bytes de contenido y los 534 bytes efectivamente transferidos, informada por las propias herramientas de desarrollador del navegador, cuantificó de manera precisa el volumen de metadatos que el usuario no percibe.

La distinción entre las opciones `-i` e `-I` de curl condujo a comprender el método HEAD y su utilidad práctica para consultar los metadatos de un recurso sin transferir su contenido. La ejecución comparativa de ambas variantes sobre el mismo recurso inexistente evidenció que el código de estado y los encabezados son idénticos, y que la única diferencia reside en la presencia del cuerpo del mensaje.

Finalmente, el análisis del registro de accesos aportó el elemento de mayor valor formativo: la constatación de que el formato combined de Ubuntu contabiliza el total de bytes enviados —encabezados incluidos— y no únicamente el cuerpo de la respuesta. La coherencia aritmética entre los valores registrados y los encabezados observados con curl permitió validar esta interpretación, poniendo de manifiesto la importancia de conocer con exactitud la definición del formato de log antes de extraer conclusiones sobre el tráfico de un servidor.

22. Anexo — Comandos utilizados en el Ejercicio 3

```
# Ejecutados desde el Símbolo del sistema de Windows (host 192.168.1.31)
C:\Windows\System32\curl.exe -i http://192.168.1.2
C:\Windows\System32\curl.exe -I http://192.168.1.2/noexiste.html
C:\Windows\System32\curl.exe -i http://192.168.1.2/noexiste.html

# Ejecutados en la máquina virtual mediante sesión SSH
sudo tail -n 20 /var/log/apache2/wwwreces_access.log
grep LogFormat /etc/apache2/apache2.conf
```

23. Ejercicio 4 — Objetivo y consideración previa

El Ejercicio 4 requiere instalar el servidor web Nginx sobre la misma máquina virtual Ubuntu, crear un bloque de servidor (Server Block) dentro del directorio `/etc/nginx/sites-available/`, habilitarlo mediante su correspondiente enlace simbólico en `sites-enabled/`, y publicar en el puerto 80 un documento HTML provisto por la cátedra. En una segunda instancia se solicita incorporar al documento un script que realice una petición HTTP GET a una API pública externa.

23.1. Conflicto de puertos entre Apache y Nginx

La instalación de Nginx sobre un sistema en el que Apache ya se encuentra en ejecución presenta un conflicto insalvable: **dos procesos no pueden asociarse simultáneamente al mismo puerto TCP sobre la misma dirección**. Como Apache ya mantenía abierto el puerto 80 desde los ejercicios anteriores, el intento de iniciar Nginx habría fallado con un error del tipo `Address already in use`.

El enunciado contempla esta situación al sugerir la utilización de un estado anterior de la máquina virtual, previo a la instalación de Apache. No obstante, dado que dicho estado no había sido preservado, se optó por una alternativa funcionalmente equivalente y de menor impacto: **detener y deshabilitar el servicio Apache** en lugar de desinstalarlo. De este modo el puerto 80 queda liberado para Nginx, mientras que la totalidad de la configuración desarrollada en los Ejercicios 1, 2 y 3 permanece intacta en el sistema de archivos y puede restablecerse en cualquier momento.

Como medida preventiva, previamente a cualquier modificación se generó una instantánea (snapshot) de la máquina virtual en VirtualBox, denominada "Apache configurado", que permite revertir el sistema al estado inmediatamente anterior a la instalación de Nginx en caso de resultar necesario.

24. Punto 1 — Instalación del servidor Nginx

24.1. Liberación del puerto 80

Antes de instalar el nuevo servidor se detuvo el servicio Apache y se deshabilitó su arranque automático:

```
sudo systemctl stop apache2
sudo systemctl disable apache2
sudo systemctl status apache2
sudo ss -tlnp | grep :80
```

El comando `stop` finaliza el proceso en ejecución, mientras que `disable` elimina el enlace simbólico de arranque automático, evitando que el servicio vuelva a iniciarse al reiniciar el sistema y ocupe nuevamente el puerto. La consulta con `systemctl status` confirmó el estado `inactive (dead)`, y la verificación con `ss` no devolvió resultado alguno, comprobando que el puerto 80 había quedado efectivamente libre.

```
Synchronizing state of apache2.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install disable apache2
Removed /etc/systemd/system/multi-user.target.wants/apache2.service.
o apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; disabled; vendor preset: enabled)
   Active: inactive (dead)
     Docs: https://httpd.apache.org/docs/2.4/

sep 07 21:02:13 ubuntu-server systemd[1]: Starting The Apache HTTP Server...
sep 07 21:02:14 ubuntu-server apachectl[693]: AH00558: apache2: Could not reliably determine the server's full
sep 07 21:02:14 ubuntu-server systemd[1]: Started The Apache HTTP Server.
sep 07 21:03:18 ubuntu-server systemd[1]: Stopping The Apache HTTP Server...
sep 07 21:03:18 ubuntu-server apachectl[1122]: AH00558: apache2: Could not reliably determine the server's full
sep 07 21:03:18 ubuntu-server systemd[1]: apache2.service: Deactivated successfully.
sep 07 21:03:18 ubuntu-server systemd[1]: Stopped The Apache HTTP Server.
```

Salida de `systemctl status apache2` en estado `inactive (dead)` y del comando `ss` confirmando que ningún proceso ocupa el puerto 80.

24.2. Comando de instalación

La instalación se realizó desde los repositorios oficiales de Ubuntu, mediante el gestor de paquetes APT:

```
sudo apt update
sudo apt install -y nginx
```

El paquete `nginx` instala el servidor junto con sus dependencias, crea la estructura de directorios de configuración en `/etc/nginx/` y registra el servicio en `systemd`, dejándolo habilitado e iniciado automáticamente al finalizar la instalación.

24.3. Verificación del servicio

```
sudo systemctl status nginx
nginx -v
```

El servicio quedó en estado `active` (`running`). Se verificó adicionalmente el funcionamiento accediendo desde el navegador del equipo anfitrión a `http://192.168.1.2`, obteniéndose la página de bienvenida por defecto "Welcome to nginx!". Este resultado confirma que Nginx tomó exitosamente el puerto 80 previamente liberado por Apache.

[illegible]

Salida de `systemctl status nginx` en estado active (running) junto con la versión devuelta por `nginx -v`.

25. Punto 2 — Creación y habilitación del Server Block

25.1. Directorio y contenido del sitio

Se creó el directorio requerido por el enunciado, denominado `wwwredesnginx`, y se generó dentro de él el archivo `index.html` con el código HTML provisto por la cátedra, reemplazando el marcador `[XXXXXXX]` por la Libreta Universitaria del alumno:

```
sudo mkdir -p /var/www/wwwreidesnginx
sudo tee /var/www/wwwreidesnginx/index.html > /dev/null <<'EOF'
<!DOCTYPE html>
<html lang="es">
<head>
```

```

<meta charset="UTF-8">
<title>Redes de Datos</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f9;
    color: #333;
    text-align: center;
    margin-top: 100px;
  }
  h1 { color: #802704; font-size: 3em; }
  h2 { color: #444; font-size: 2em; }
</style>
</head>
<body>
  <h1>Bienvenido a Nginx - Redes de Datos | LU: 58009</h1>
  <h2>HTTP con Nginx</h2>
</body>
</html>
EOF

sudo chown -R www-data:www-data /var/www/wwwredesnginx
sudo chmod -R 755 /var/www/wwwredesnginx

```

Se mantuvo el usuario `www-data` como propietario del contenido. Cabe señalar que, al igual que Apache en Ubuntu, Nginx ejecuta sus procesos de trabajo (worker processes) bajo ese mismo usuario, según se define en la directiva `user` del archivo `/etc/nginx/nginx.conf`.

25.2. Definición del Server Block

Se creó el archivo de configuración dentro del directorio solicitado por el enunciado. Corresponde destacar que, a diferencia de Apache, **los archivos de sitio de Nginx no llevan extensión `.conf`**, ya que la inclusión se realiza mediante un comodín sobre la totalidad del directorio:

```

sudo nano /etc/nginx/sites-available/wwwredesnginx

server {
    listen 80;
    listen [::]:80;

    root /var/www/wwwredesnginx;
    index index.html;

    server_name wwwredesnginx.local;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/wwwredesnginx_access.log;
    error_log /var/log/nginx/wwwredesnginx_error.log;
}

```

Significado de cada directiva empleada:

Directiva	Función
listen 80	Puerto de escucha del bloque. A diferencia de Apache, Nginx no requiere un archivo separado equivalente a <code>ports.conf</code> : la propia directiva abre el socket.
listen [::]:80	Declaración análoga para el protocolo IPv6.

Directiva	Función
root	Directorio raíz desde el cual se sirven los archivos. Equivale al DocumentRoot de Apache.
index	Archivo a entregar cuando la petición apunta a un directorio.
server_name	Nombre que identifica al bloque, comparado con el encabezado Host de la petición.
location /	Bloque que define el tratamiento de las peticiones cuya ruta coincide con el patrón indicado.
try_files \$uri \$uri/ =404	Intenta resolver la petición como archivo, luego como directorio, y de no encontrar ninguno devuelve un código 404.
access_log / error_log	Rutas de los registros propios de este bloque, análogas a CustomLog y ErrorLog en Apache.

```
redes@ubuntu-server:~$ cat /etc/nginx/sites-available/wwwredesnginx
server {
    listen 80;
    listen [::]:80;

    root /var/www/wwwredesnginx;
    index index.html;

    server_name wwwredesnginx.local;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/wwwredesnginx_access.log;
    error_log /var/log/nginx/wwwredesnginx_error.log;
}
```

Contenido del archivo /etc/nginx/sites-available/wwwredesnginx obtenido con cat.

25.3. Habilitación mediante enlace simbólico

Nginx replica el esquema de directorios sites-available y sites-enabled que utiliza Apache en Ubuntu, pero **no provee herramientas equivalentes a a2ensite y a2dissite**. La habilitación de un sitio se realiza creando manualmente el enlace simbólico, tal como solicita expresamente el enunciado:

```
sudo ln -s /etc/nginx/sites-available/wwwredesnginx /etc/nginx/sites-enabled/
sudo unlink /etc/nginx/sites-enabled/default
ls -l /etc/nginx/sites-enabled/
```

El listado del directorio confirmó la creación del enlace:

```
lrwxrwxrwx 1 root root 40 sep  7 21:10 wwwredesnginx ->
    /etc/nginx/sites-available/wwwredesnginx
```

El primer carácter l de los permisos identifica la entrada como enlace simbólico, y la notación con flecha indica el destino al que apunta.

Se eliminó asimismo el enlace del sitio default que Nginx habilita durante su instalación. Dicho bloque incorpora el modificador default_server en su directiva listen, lo que lo convierte en el destinatario de toda petición cuyo encabezado Host no coincida con ningún server_name definido, como ocurre al acceder

mediante la dirección IP. De no haberse retirado, las peticiones a `http://192.168.1.2` habrían continuado siendo atendidas por la página de bienvenida en lugar del sitio configurado.

```
redes@ubuntu-server:~$ ls -l /etc/nginx/sites-enabled/
total 0
lrwxrwxrwx 1 root root 40 sep  7 21:10 wwwredesnginx -> /etc/nginx/sites-available/wwwredesnginx
```

Salida del comando `ls -l /etc/nginx/sites-enabled/` mostrando el enlace simbólico `wwwredesnginx` apuntando al archivo de `sites-available`.

25.4. Validación y aplicación de la configuración

```
sudo nginx -t
sudo systemctl reload nginx
```

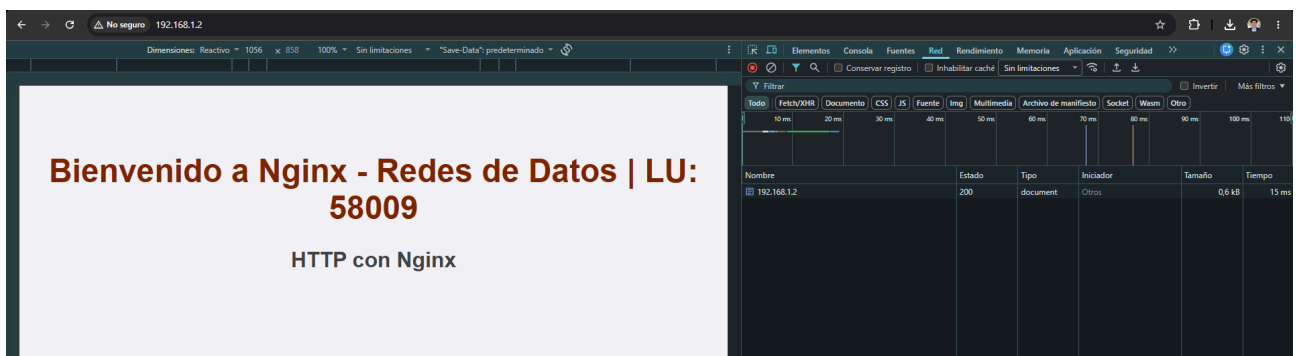
La validación devolvió el siguiente resultado:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

El comando `nginx -t` cumple la misma función que `apache2ctl configtest`: verifica la sintaxis de la configuración sin aplicarla, permitiendo detectar errores antes de recargar el servicio. Se comprobó finalmente desde el navegador que la dirección `http://192.168.1.2` entregaba el documento configurado, con el encabezado en color y la Libreta Universitaria visible.

```
redes@ubuntu-server:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Terminal mostrando el resultado de `nginx -t` con los mensajes "syntax is ok" y "test is successful".



Navegador mostrando `http://192.168.1.2` con la página "Bienvenido a Nginx - Redes de Datos | LU: 58009".

25.5. Comparación entre Apache y Nginx

La implementación de ambos servidores sobre el mismo sistema permitió contrastar sus esquemas de configuración:

Aspecto	Apache	Nginx
Unidad de configuración de sitio	VirtualHost	Server Block
Declaración del puerto	Listen en ports.conf, global	listen dentro del propio bloque server
Directorio raíz	DocumentRoot	root

Aspecto	Apache	Nginx
Extensión de los archivos de sitio	Requiere .conf	Sin extensión
Habilitación de sitios	a2ensite / a2dissite	Enlace simbólico manual con ln -s
Validación de sintaxis	apache2ctl configtest	nginx -t
Configuración por directorio	Admite archivos .htaccess	No los admite; toda la configuración es centralizada
Usuario de ejecución	www-data	www-data

26. Punto 3 — Consumo de una API pública externa

26.1. Modificación del documento HTML

Se reemplazó el contenido del archivo `/var/www/wwwredesnginx/index.html` por la versión provista en el enunciado, que incorpora un contenedor destinado a mostrar el resultado de la consulta y un script que ejecuta una petición HTTP GET mediante la interfaz `fetch` hacia la API pública `api.ipify.org`, la cual devuelve en formato JSON la dirección IP pública del cliente que la consulta.

```
fetch('https://api.ipify.org?format=json')
  .then(response => {
    if (!response.ok) throw new Error('Error en la respuesta HTTP');
    return response.json();
  })
  .then(data => {
    document.getElementById('api-result').innerText =
      'La IP Publica del cliente es: ' + data.ip;
  })
  .catch(error => {
    document.getElementById('api-result').innerText =
      'Fallo en la conexion a la API.';
    console.error(error);
  });
```

Aplicada la modificación se recargó el servicio:

```
sudo systemctl reload nginx
```

26.2. Incidencia detectada durante la implementación

En el primer intento de verificación el contenedor permaneció indefinidamente con el texto inicial "Realizando petición HTTP...", sin mostrar ni el resultado esperado ni el mensaje de error previsto en el bloque `catch`.

El diagnóstico se orientó inicialmente hacia un problema de conectividad, hipótesis que fue descartada al inspeccionar la pestaña Red de las herramientas de desarrollador: **no figuraba ninguna petición dirigida a `api.ipify.org`**. Si el script se hubiera ejecutado y la solicitud hubiese fallado por causas de red, la petición aparecería registrada con su correspondiente estado de error. Su ausencia total indicaba que el código nunca llegó a ejecutarse.

La causa se localizó al inspeccionar el archivo con `tail`. Al copiar el código desde el documento PDF del enunciado, los saltos de línea propios del formato del documento se habían incorporado literalmente dentro de las cadenas de texto del script, quedando por ejemplo:

```
if (!response.ok) throw new Error('Error  
en la respuesta HTTP');
```

En JavaScript, una cadena delimitada por comillas simples **no admite saltos de línea literales en su interior**. Al encontrar el fin de línea antes de la comilla de cierre, el analizador sintáctico genera un error de tipo `SyntaxError: Invalid or unexpected token` y descarta la ejecución del bloque `<script>` en su totalidad. Este comportamiento explica de manera precisa el síntoma observado: el navegador renderizó el HTML correctamente —incluido el texto inicial del contenedor— pero jamás ejecutó la petición, motivo por el cual tampoco se activó el bloque `catch` encargado de reportar fallos.

La solución consistió en reescribir el archivo manteniendo cada cadena de texto en una única línea, tal como se encuentra en el código original del enunciado. Tras aplicar la corrección y recargar el navegador con caché deshabilitada, el script se ejecutó correctamente.

Se deja constancia de esta incidencia por su valor formativo: evidencia que un error de sintaxis en JavaScript no degrada parcialmente la funcionalidad sino que anula por completo el bloque afectado, y que la ausencia de una petición en el panel de red constituye un indicio diagnóstico distinto del de una petición fallida.

26.3. Verificación y análisis del resultado

La página se cargó correctamente mostrando el mensaje "La IP Pública del cliente es: 181.86.17.240". El panel de red de las herramientas de desarrollador registró dos peticiones:

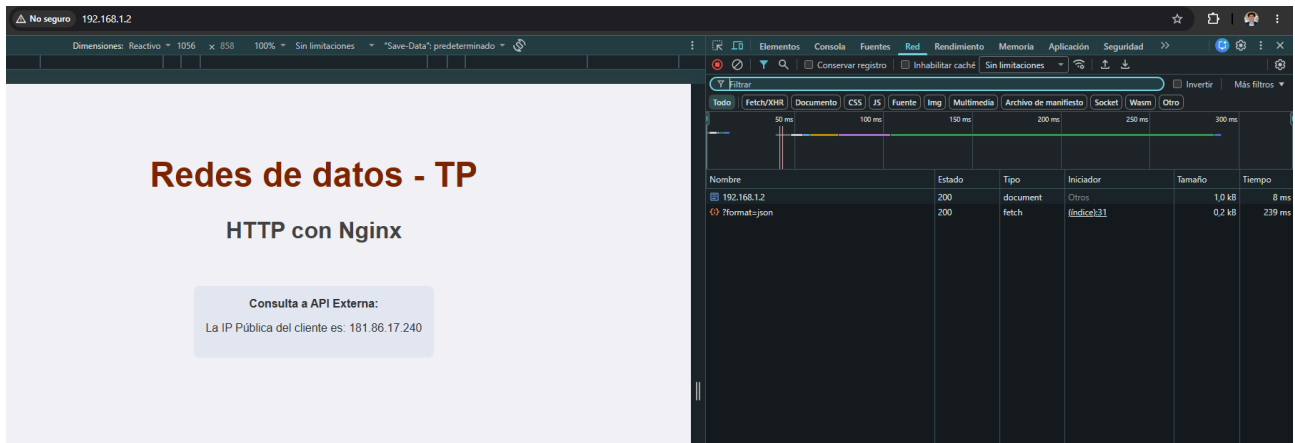
Petición	Estado	Tipo	Iniciador	Tiempo
192.168.1.2	200	document	Otros	8 ms
?format=json	200	fetch	(índice):31	239 ms

El análisis de estos datos permite extraer varias conclusiones relevantes:

- **Tipo de petición.** La primera figura como `document`, correspondiente a la carga inicial del HTML servido por Nginx. La segunda aparece clasificada como `fetch`, lo que indica que no fue solicitada por el navegador durante el análisis del documento sino disparada programáticamente por el código JavaScript.
- **Iniciador.** El valor `(índice):31` señala que la petición se originó en la línea 31 del documento HTML, es decir en la instrucción `fetch`. Constituye la evidencia directa de que el script se ejecutó efectivamente.
- **Tiempos de respuesta.** La diferencia entre 8 ms y 239 ms resulta significativa: la primera petición se resuelve dentro de la red local, contra un servidor situado a un solo salto; la segunda debe atravesar el router doméstico, la infraestructura del proveedor de servicios de internet y alcanzar un servidor remoto, con la consiguiente latencia adicional.
- **Naturaleza de la dirección obtenida.** El valor devuelto no corresponde ni a 192.168.1.31 (equipo anfitrión) ni a 192.168.1.2 (servidor), sino a la dirección pública asignada por el proveedor al router de la red. Ello se debe a que la petición sale a internet atravesando el mecanismo de traducción de direcciones (NAT) del router, que reemplaza la dirección privada de origen por su propia dirección pública. En consecuencia, la API observa esa dirección y no la del equipo que originó la consulta.

Aspecto conceptual destacado: la petición a la API **no la realiza el servidor Nginx**, sino el navegador del cliente. El servidor se limitó a entregar el documento HTML; la ejecución del script y la conexión hacia `api.ipify.org` ocurren íntegramente en el equipo anfitrión. Esto explica que la dirección informada sea la

pública de la red del cliente, e implica además que el correcto funcionamiento de esta prueba depende de la conectividad a internet del equipo que visualiza la página, no de la de la máquina virtual.



Navegador mostrando la página con el resultado "La IP Pública del cliente es: ..." junto al panel de red de las herramientas de desarrollador.

27. Conclusiones del Ejercicio 4

Se cumplieron íntegramente los requerimientos planteados. Nginx quedó instalado y operativo, con un Server Block propio creado en sites-available, habilitado mediante enlace simbólico en sites-enabled y publicando en el puerto 80 el documento solicitado, incluida la consulta a la API externa funcionando correctamente.

La resolución del conflicto de puertos entre ambos servidores constituyó el primer aprendizaje del ejercicio: la imposibilidad de que dos procesos compartan un mismo puerto sobre la misma dirección es una restricción del sistema operativo y no del software de servidor, y su comprensión resulta indispensable para administrar entornos donde conviven múltiples servicios.

La comparación entre ambos servidores permitió advertir que, si bien comparten conceptos equivalentes —raíz documental, nombre de sitio, puerto de escucha, registros diferenciados—, difieren en aspectos operativos relevantes. Nginx concentra la declaración del puerto dentro del propio bloque de servidor, prescinde de herramientas auxiliares para la habilitación de sitios y no admite configuración distribuida por directorio mediante archivos .htaccess, lo que impone una administración centralizada pero elimina el costo de rendimiento asociado a la consulta de dichos archivos en cada petición.

Finalmente, la incidencia detectada con el script de consumo de la API resultó ser el episodio de mayor valor formativo del ejercicio. El diagnóstico exigió descartar sucesivamente las hipótesis de fallo de conectividad y de error de la API para arribar a la causa real, un error de sintaxis introducido al transcribir el código desde el documento del enunciado. La ausencia de la petición en el panel de red —en contraposición a una petición registrada con estado de error— fue el indicio determinante que permitió reorientar el análisis hacia el código y no hacia la red.

28. Anexo — Comandos utilizados en el Ejercicio 4

```
# Liberacion del puerto 80
sudo systemctl stop apache2
sudo systemctl disable apache2
sudo systemctl status apache2
sudo ss -tlnp | grep :80

# Instalacion de Nginx
sudo apt update
sudo apt install -y nginx
sudo systemctl status nginx
nginx -v

# Creacion del sitio
sudo mkdir -p /var/www/wwwredesnginx
sudo nano /var/www/wwwredesnginx/index.html
sudo chown -R www-data:www-data /var/www/wwwredesnginx
sudo chmod -R 755 /var/www/wwwredesnginx

# Server Block y habilitacion
sudo nano /etc/nginx/sites-available/wwwredesnginx
sudo ln -s /etc/nginx/sites-available/wwwredesnginx /etc/nginx/sites-enabled/
sudo unlink /etc/nginx/sites-enabled/default
ls -l /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

# Diagnostico del script de la API
grep -c "fetch" /var/www/wwwredesnginx/index.html
tail -30 /var/www/wwwredesnginx/index.html
```